Empirical Software Engineering (2026) 31:140
https://doi.org/10.1007/s10664-026-10868-6

Leveraging large language models for sentiment analysis in GitHub pull request discussions

Daniel Coutinho[1] · **Breno Braga**[1] · **Theo Canuto**[1] · **Juliana Alves Pereira**[1] · **Wesley K. G. Assunção**[2] · **Igor Steinmacher**[3] · **Marco Gerosa**[3] · **Alessandro Garcia**[1]

Received: 14 August 2025 / Accepted: 21 April 2026 / Published online: 9 May 2026

Abstract
Social coding platforms like GitHub facilitate collaborative software development through pull requests (PRs), which generate discussions that significantly impact code quality, requirements, and design. Such conversations become a rich source of insights for improving development practices and predicting project outcomes and are subject to several human aspects that have been linked to code quality and PR acceptance. Sentiment analysis is one of the many ways to try to understand these human aspects. However, PR discussions are multifaceted, often involving technical jargon and aspects which limits the utility of general-purpose sentiment analysis tools. This has led to the creation of SE-specific tools, but recent studies have also observed that they demonstrate limited effectiveness. Thus, this study explores the potential of using large language models (LLMs) for this purpose, given their enhanced contextual understanding and ability to process technical language. We evaluated ten LLMs across proprietary and open-source categories, using two complementary datasets: a curated Gold dataset and the PRemo dataset, which captures real-world PR discussions. The models were assessed under zero-shot, few-shot and chain-of-thought prompting techniques on 8,913 messages. In addition, we establish baselines by evaluating fine-tuned transformer-based models. Results show that GPT-4o achieved the highest overall performance across the LLMs, though smaller models, such as Mistral Small and Deepseek-R1 32B delivered competitive results. Transformer-based models achieved excellent performance on the Gold dataset but exhibited degradation on the PRemo dataset. Finally, we conducted a qualitative analysis of misclassified instances, revealing recurring challenges related to technical terminology, sentiment-charged keywords, message length, and contextual ambiguity. These findings suggest that model selection should balance performance requirements against practical constraints, rather than defaulting to the largest available models.

Keywords Large language models · Human aspects · Sentiment analysis · Prompt engineering

Communicated by: Sebastian Baltes.

Extended author information available on the last page of the article

1 Introduction

Modern software development relies on collaborative platforms such as GitHub[1] and GitLab[2], which provide social features for enabling asynchronous and distributed development processes (Tantisuwankul et al. 2019; Kalliamvakou et al. 2015). Pull requests (PRs), a core feature of these platforms, enable developers to propose code changes and engage in structured discussions through integrated messaging systems (Rahman and Roy 2014). These discussions include both code-specific comments targeting particular code snippets and broader conversational threads addressing project-wide concerns (Zhang et al. 2022). Recent research has demonstrated that these broader discussions often involve requirements analysis (Mannan et al. 2020) and architectural design decisions establishing their significance as determinants of long-term software quality and maintainability (Barbosa et al. 2023).

Human factors in this communication process can significantly impact the software development process, influencing the introduction of bugs to the acceptance of a PR (Tsay et al. 2014a; El Mezouar et al. 2019). Consequently, understanding how this communication occurs and how it can be most effective becomes a relevant target for practitioners and researchers (Guzman and Bruegge 2013). While several analytical approaches exist for examining these interactions, sentiment analysis of developer communications has received considerable attention due to its demonstrated associations with various aspects of software development (Guzman et al. 2014; Calefato et al. 2018). In this paper, we define *Sentiment* as the analytical target of *Sentiment Analysis*, which involves classifying the overall subjective polarity[3] of textual communication into categories such as positive, negative, or neutral (Dang et al. 2020).

As general-purpose sentiment analysis tools typically underperform when applied to software engineering-related texts (Novielli et al. 2018), several software engineering-specific sentiment analysis tools have been developed (Islam and Zibran 2018a; Calefato et al. 2017). However, recent evaluations indicate that these specialized tools not only suffer from practical deployment challenges, including broken dependencies, inadequate documentation, and requirements for obsolete software versions, but also deliver unsatisfactory performance when analyzing PR discussions. In this work, we refer to performance as the precision, recall, and F1-score metrics achieved by a specific tool or model.

This performance gap presents significant opportunities for the use of large language models (LLMs), which demonstrate robust capabilities to process domain-specific language patterns and technical terminology typical of software development communications (Brown et al. 2020; Gururangan et al. 2020). Nevertheless, the practical effectiveness of LLMs in sentiment analysis tasks for PR discussions remains largely unexplored, particularly regarding comparative performance across model architectures and the influence of various prompt engineering strategies on sentiment classification accuracy, and the extent to which results generalize across different types of evaluation datasets.

This study assesses the effectiveness and practical utility of employing LLMs for sentiment analysis in PR discussions on GitHub, the world's leading collaborative development platform. We evaluate the sentiment classification capabilities of ten different LLMs

[1] https://github.com/

[2] https://gitlab.com/

[3] Not to be confused with fine-grained emotion detection, which we do not analyze in this work.

applied to software engineering discussions using two complementary datasets: a curated Gold standard dataset (Novielli et al. 2020), which captures commit and PR comments, and the PRemo dataset (Coutinho et al. 2024), which captures PR discussion messages. Both datasets total 8,913 manually labeled messages.

We further investigate how various prompt engineering techniques, such as few-shot learning and chain-of-thought reasoning, impact classification accuracy across both datasets. By comparing proprietary and open-source models, we analyze trade-offs related to deployment feasibility, model size, and cost efficiency. Our findings provide insights into whether LLMs, especially when enhanced by prompt engineering, effectively manage the specific language and sentiment patterns characteristic of developer communications. The results offer empirical guidance for practitioners and researchers on model selection aligned with performance requirements, budget constraints, and deployment considerations, thereby advancing robust and scalable sentiment analysis solutions for collaborative software development contexts.

As a comparison baseline, we also evaluated four widely adopted pre-trained transformer models for sentiment classification, following the experimental setup proposed by Zhang et al. (2020). Specifically, we fine-tuned BERT, XLNet, RoBERTa, and ALBERT using their standard base configurations to establish a strong non-LLM reference grounded in prior software engineering research. This baseline allows us to contextualize the performance achieved by LLMs and prompt engineering techniques relative to established transformer-based approaches that have been widely used for sentiment analysis in software engineering.

Our findings indicate that all the models evaluated exceeded traditional sentiment analysis tools, based on benchmarks from the literature (Coutinho et al. 2024). Specifically, results highlight significant variability in performance across different sentiment categories (positive, negative, neutral) and across datasets. The proprietary GPT-4o model achieved the strongest overall performance, particularly when prompt engineering techniques such as chain-of-thought reasoning were applied, yielding substantial recall improvements on the PRemo dataset. Meanwhile, open-source models, including Mistral, Llama 3.1, and Deepseek-R1, demonstrated competitive performance, with clear cost advantages due to their relatively smaller size.

We found that no single LLM excelled universally, underscoring the importance of selecting models based on specific task requirements, cost considerations, and implementation constraints (e.g., hardware availability). Beyond aggregate performance differences, a qualitative analysis of misclassified instances reveals that errors frequently stem from domain-specific language, sentiment-charged technical terminology, message structure, and contextual ambiguity rather than simple polarity confusion. These findings suggest that LLMs, particularly when combined with prompt engineering, offer a promising approach for sentiment analysis in software development, while also highlighting persistent challenges in complex developer communication. As such, the contributions of this work include:

(i) An empirical analysis of the performance of ten diverse LLMs on sentiment classification of developer communications in GitHub PR discussions across two different datasets.
(ii) A comparative evaluation of four widely adopted pre-trained transformer models (BERT, XLNet, RoBERTa, and ALBERT), fine-tuned following a process used by

prior software engineering research, serving as a strong non-LLM baseline for sentiment classification.

(iii) A detailed discussion based on empirical results regarding practical considerations for selecting and deploying LLMs (e.g., proprietary vs. open-source models) for sentiment analysis and related tasks.

(iv) An examination of how prompt engineering techniques can be applied in sentiment analysis and influence its performance.

(v) A qualitative analysis of sentiment misclassifications that identifies recurring linguistic, structural, and contextual factors, such as technical terminology, sentiment-charged keywords, and message length, that challenge LLM-based sentiment interpretation in developer communication.

2 Background

2.1 Sentiment Analysis

Sentiment analysis, often referred to as opinion mining, is a natural language processing (NLP) task concerned with identifying subjective information in text (Hasan et al. 2024). While some approaches focus on detecting specific emotions (e.g., joy, anger), in this work sentiment analysis is defined strictly as polarity-based classification (positive, negative, or neutral) (Pang et al. 2008).

Traditionally, sentiment analysis relied on either rule-based or learning-based approaches. Rule-based approaches apply predefined linguistic rules and lexicons to interpret sentiment, whereas learning-based approaches (*e.g.*, Naive Bayes and Support Vector Machines) learn from labeled data to classify sentiments (Liu 2022). Although these traditional methods are effective on some use cases, they often struggle with complex contexts, including sarcasm and idiomatic expressions (Liu 2022).

In software engineering, team sentiment directly affects productivity and product quality (Graziotin et al. 2014, 2015). Analysing textual interactions provides project managers with actionable feedback. Early detection of negative patterns enables proactive issue resolution, mitigating conflicts and reducing stress. Moreover, fostering a positive atmosphere boosts morale, fuels innovation and creativity (Herrmann and Klünder 2021). By systematically analyzing and managing team members' sentiment, teams strengthen communication and collaboration, enhance well-being and drive project success (Ain et al. 2017; Herrmann and Klünder 2021).

2.2 LLMs in Sentiment Analysis

The rise of LLMs, such as transformer-based models like BERT (Devlin et al. 2018) and GPT (Brown et al. 2020), has revolutionized NLP. These LLMs are mostly built on the transformer architecture (Vaswani et al. 2017), which uses self-attention mechanisms to capture relationships between words in a text, regardless of their position. This enables them to grasp context and nuance far more effectively than earlier approaches, such as bag-of-words or basic word embeddings. One of the core strengths of LLMs is their ability to generalize across domains and tasks with minimal fine-tuning, making them exceptionally powerful for tasks such as sentiment analysis, where subtle variations in language and context are critical (Zhan et al. 2024).

Recent work (Imran et al. 2024) has started to probe whether these general-purpose models transfer to the idiosyncratic language of software projects. Recent studies evaluated ChatGPT, GPT-4 and the open-source flan-alpaca in a zero-shot setting across three developer-centric corpora (GitHub, Stack Overflow and Jira), finding that GPT-4 matches or surpasses customized software engineering classifiers for emotion labelling.

Advanced LLMs, like GPT-4 (Brown et al. 2020) are pre-trained on massive datasets, enabling them to generate human-like responses and perform a wide range of NLP tasks. By applying techniques such as prompt engineering, these models can be further refined to capture fine-grained nuances and complex contexts, enhancing both the accuracy and robustness of their analyses (Niimi 2024; Xing 2024; Zhan et al. 2024; Wei et al. 2022).

2.3 Prompt Engineering

Prompt engineering refers to the process of designing specific input prompts to elicit the desired responses from LLMs. Recent studies (Zhou et al. 2022; Xing 2024; Wei et al. 2022; Zhang et al. 2023b) demonstrate that well-crafted prompts can significantly boost LLM performance in sentiment analysis by enhancing model interpretability and relevance. For instance, incorporating additional contextual details in prompts allows models to refine their responses, leading to more precise sentiment classification and extraction of nuanced emotional cues from text. Xing (2024) proposed a framework where heterogeneous LLM agents collaborate, discussing specific sentiment classification tasks to arrive at a more robust analysis with minimal data. Additionally, Wei et al. (2022) emphasizes the value of prompt engineering for structured information extraction tasks. Techniques such as few-shot learning, where a few task examples are included within the prompt, allow the model to adapt to new tasks without extensive fine-tuning (Brown et al. 2020).

Furthermore, studies reveal that LLMs are surprisingly brittle to small prompt variations. Zhu et al. (2023)'s PromptRobust benchmark systematically applies character-, word-, sentence-, and semantic-level perturbations to prompts across 8 NLP tasks, including SST-2 sentiment analysis. They find that word-level prompt attacks (e.g., minor typos, synonym swaps) induce an average 39% drop in accuracy across tasks - and in the case of SST-2, accuracy plunges from 96.1% to 62% under adversarial prompts. Another study (Li et al. 2023) introduces a Performance Drop Rate (PDR) metric to quantify how much QA models' answer accuracy falls when benign yet misleading instructions are injected into the context. Evaluating models such as GPT-4, they observe PDRs of 15-25%, meaning up to a quarter of correct responses are lost purely due to extraneous "distractor" questions embedded in the prompt.

3 Related Work

In this section, we present related work grouped into four key areas: (i) traditional and modern approaches to sentiment analysis in software engineering; (ii) sentiment analysis specifically in GitHub, with a focus on PR discussions; (iii) the role of prompt engineering for improving sentiment analysis in mixed technical-natural language settings; and (iv) the use of LLMs for sentiment analysis in software engineering, highlighting how our work builds on and advances prior efforts.

3.1 Sentiment Analysis in SE

Traditional sentiment analysis techniques, such as lexicon-based approaches and classical machine learning models (*e.g.*, Thelwall et al. 2012; Kolchyna et al. 2015; Islam and Zibran 2018b), have several limitations. Lexicon-based approaches often struggle to capture the contextual meaning of words, especially when dealing with domain-specific language or idiomatic expressions (Liu 2022). Similarly, classical machine learning models are typically less effective in handling ambiguous or context-dependent language without extensive feature engineering (Pang et al. 2008).

These limitations become especially pronounced in the field of software engineering, where communication often includes technical jargon, informal language, and non-textual content such as code snippets and emojis (Lin et al. 2018). LLMs, such as GPT-4, with their ability to model long-range dependencies and capture nuanced meaning, offer a significant improvement over traditional methods in performing sentiment analysis (Zhan et al. 2024). GPT-4 outperforms smaller models like BERT or seBERT in scenarios with limited labeled data due to its flexibility in handling diverse linguistic inputs with minimal training (Zhang et al. 2023b; Shafikuzzaman et al. 2024). This practical advantage makes GPT-4 highly suitable for software engineering, where manually labeling data is time-consuming and labor-intensive.

3.2 Sentiment Analysis in GitHub

Many studies in the field of sentiment analysis within software engineering have primarily focused on aspects such as developer communications in issue trackers, forums, or chat rooms (Obaidi et al. 2022; Calefato et al. 2017). On one of these platforms, GitHub, one critical element that is often looked at is PRs. Previous works often look at PRs only as a venue for studying code review (Tsay et al. 2014b), overlooking that a PR also contains a general discussion thread, which is often not directly related to a specific excerpt of source code. Recent studies suggest that this thread (which we call PR discussion) is where higher-level design decisions about the source code are often made (Viviani et al. 2019; Barbosa et al. 2023), as they are a place for discussing higher-level topics, relating to the whole change being proposed as part of the PR.

Previous studies (e.g., Calefato et al. 2017; Ortu et al. 2016) indicate that emotions expressed in software development discussions can affect the quality of decisions and overall productivity. Thus, incorporating sentiment analysis on PR discussions allows for a richer understanding of the social dynamics at play and can inform interventions to improve collaboration within open-source software projects. While some tools are specifically tailored for sentiment analysis in software engineering discussions (Calefato et al. 2018), their effectiveness in the context of PR discussions remains limited. In a recent study (Coutinho et al. 2024), we showed that existing tools, although usable on PR discussions, achieve relatively poor performance in this setting. Therefore, we hypothesize that by utilizing LLMs, we can potentially have a more nuanced understanding of the technical language that is employed in these discussions.

3.3 Prompt Engineering for Sentiment Analysis

Prompt Engineering strategies become especially valuable when handling complex, unstructured data sources, such as discussions in collaborative development platforms, where understanding context and disambiguating language are essential for accu-

rate sentiment interpretation, especially in cross-platform scenarios (*e.g.*, GitHub and Jira) (Novielli et al. 2020). Specifically, our analysis targets sentiment in the context of PR discussions. Whereas previous studies (Zhu et al. 2023; Li et al. 2023) establish an open-book question-answering benchmark for assessing instruction-following robustness under prompt-injection attacks - injecting adversarial instructions into retrieved web contexts for datasets such as NaturalQuestions, TriviaQA, SQuAD and HotpotQA -, those studies do not tackle the challenge of prompts that mix code diffs, natural-language comments and technical jargon.

3.4 LLMs for Sentiment Analysis in SE

Recent studies have observed that it is feasible to utilize LLMs to perform sentiment analysis tasks in the context of software engineering (Imran et al. 2024; Zhang et al. 2023b). Additionally, one of these studies found that especially bigger LLMs can exhibit state-of-the-art performance (Zhang et al. 2023b). The primary novelty of this work, when compared to these previous works, lies in systematically evaluating prompt engineering techniques and newer LLMs (both closed and open-source) for sentiment analysis in software engineering. Unlike prior studies, we explore a diverse set of models and configurations emphasizing practical usability, instead of feasibility.

This study distinguishes itself from our previous works in three key ways. First, we provide a systematic and comparative evaluation of a diverse set of ten LLMs, spanning proprietary and open-source models with varying sizes and deployment constraints, using two datasets that specifically use GitHub data. Second, rather than treating prompting as a fixed design choice, we explicitly investigate the impact of multiple prompt engineering strategies, including one-shot, few-shot, and chain-of-thought prompting, analyzing how these techniques affect performance across different model architectures. Third, we complement quantitative evaluation with a qualitative analysis of misclassifications, offering insight into the limitations that persist even in the best-performing configurations.

Beyond extending the experimental setup, this work contributes new empirical and methodological insights for both software engineering research and practice. For researchers, our findings clarify how model capacity, prompting strategy, and dataset characteristics jointly shape sentiment classification performance, highlighting the limits of zero-shot evaluation and the risks of relying solely on aggregate metrics. For practitioners, the results provide concrete guidance on model and prompt selection under realistic constraints, demonstrating that no single configuration is universally optimal and that sentiment analysis in PR discussions should be treated as a decision-support tool rather than an automated oracle. Together, these contributions advance understanding of when and how LLM-based sentiment analysis can be reliably applied in real-world software engineering contexts.

4 Study Design

This study aims *to investigate the effectiveness of LLMs in performing sentiment analysis in PR discussions on GitHub*. Given the specialized language and sentiment nuances in developer communications, our goal is to determine whether LLMs can achieve better accuracy than traditional sentiment analysis tools. We also aim to understand how prompt

engineering might improve their performance. We chose to compare several models (and their various variations, in terms of model complexity) that cover a wide range of requirements, such as open-source versus proprietary, local versus cloud-hosted, and higher versus lower hardware requirements.

4.1 Research Questions

Based on our aim, this study is guided by two research questions (RQs), as follows:

> **RQ$_1$**: To what extent can LLMs be utilized for executing sentiment analysis tasks?

To answer this question, we compare how a set of LLMs currently available, both proprietary and open-source, perform in sentiment analysis tasks. This comparative analysis also included a focus on trade-offs relevant to practical deployment, such as the need for hardware resources or reliance on cloud infrastructure. Although the models were evaluated in terms of performance metrics (*i.e.*, precision, recall, and F1-score), we also provide actionable insights into the advantages and limitations of each type of model based on deployment needs and resource constraints.

> **RQ$_2$**: How does the usage of prompt engineering techniques affect the performance of LLMs in sentiment analysis tasks?

A common strategy to optimize the performance of LLMs is to use prompt engineering techniques, which are known to achieve better results in certain tasks. Therefore, we experimented with several prompt engineering techniques that are suitable for the task of software engineering, making sure that we effectively and fairly evaluate the models. We also investigated whether these techniques could be a way to bridge the performance gap between smaller/cheaper and larger/expensive models. Moreover, we qualitatively evaluated why the best-performing model misclassified certain messages to understand its limitations.

4.2 Study Steps

Our study consists of five steps, which are illustrated in Fig. 1. They are also described as follows.

1) Dataset Selection. For this study, we selected two datasets. First, the PRemo dataset (Coutinho et al. 2025b), which was developed by some of the same authors of this paper. We opted to use this dataset for two key reasons. First, to the best of our knowledge, PRemo is the only publicly available dataset that specifically focuses on sentiment and emotions within the context of GitHub PR discussions, which is the direct focus of our investigation. Second, this dataset has been previously used as a benchmark to evaluate traditional sentiment analysis tools (Coutinho et al. 2024), which provides a strong baseline for our current study on LLMs (and allows us to directly compare both studies).

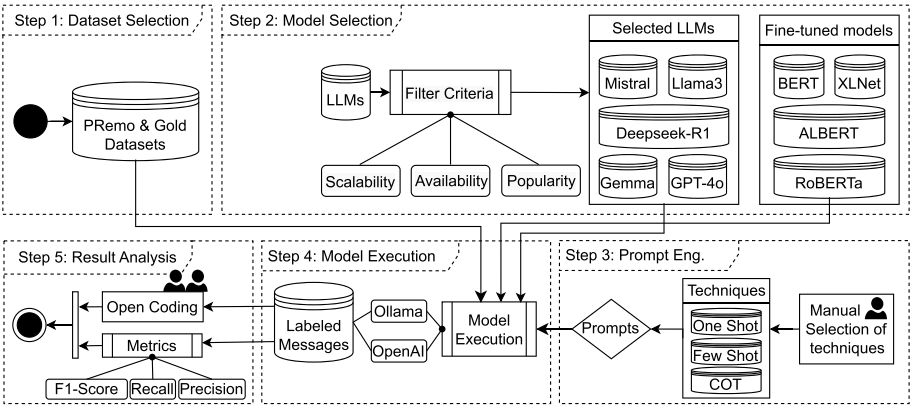

Fig. 1 Overview of the study steps

The dataset comprises 521 (≈29%) messages labeled as positive, 432 (≈24%) as negative, and 838 (≈47%) as neutral. While the distribution is not perfectly balanced, we followed the reasoning from previous literature suggesting that a sentiment distribution reflecting the original data source is often preferable to an artificially balanced one for maintaining real-world validity (Obaidi et al. 2022).

Furthermore, PRemo's construction methodology involved a meticulous process aimed at reducing subjectivity, detailed in Coutinho et al. (2025b), which includes: (i) a triple validation procedure, where each of the 1,791 messages was independently labeled by three experts; (ii) a multi-disciplinary team of 19 evaluators, comprising both software engineers and neuroscientists to ensure a balanced perspective; and (iii) a two-pass labeling system where messages were evaluated first in isolation and then with their full conversational context, enhancing the quality of the final labels. This rigorous process, grounded in an established psychological emotion model, makes PRemo a uniquely suitable and high-quality resource for our evaluation.

Second, we utilized the GitHub gold standard dataset developed by Novielli et al. (2020), which serves as a widely recognized benchmark for sentiment analysis in software engineering. This dataset was selected to complement PRemo by providing a cross-platform perspective, ensuring our evaluation of LLMs is not limited to a single data source. Throughout this work, we call this dataset the Gold dataset.

The dataset comprises 7,122 manually labeled comments from pull requests and commits, with a distribution of 2,013 (≈28%) positive, 2,087 (≈29%) negative, and 3,022 (≈ 43%) neutral messages. This distribution is similar to the one observed in PRemo, and while using this dataset, we have maintained this original distribution.

Furthermore, the construction of this dataset also followed a rigorous validation protocol, which involved: (i) an initial manual annotation by three researchers; (ii) a conflict resolution phase where disagreements were discussed until a consensus was reached; and (iii) the use of a coding guide specifically tailored to identify sentiment in technical discourse. The similarities to PRemo's construction make it an ideal companion to it for comparing if the results we observe are reproducible across other independent scenarios (and in this case, datasets.)

Table 1 Overview of the large language models evaluated in this work

Model Name	Parameters	Release Date	Provider	Quantization
GPT 4o	-	May 13, 2024	OpenAI	-
GPT 4o mini	-	July 18, 2024		-
Mistral NeMo	12 billion	June 18, 2024	Ollama	Q4_0
Mistral Small	22 billion	September 17, 2024	Ollama	Q4_0
Gemma 2	9 billion	June 27, 2024	Ollama	Q4_0
	27 billion			
Llama3.1	8 billion	July 23, 2024	Ollama	Q4_0
	70 billion			
Deepseek-R1	8 billion	January 20, 2024	Ollama	Q4_0
	32 billion			

2) Model Selection. For selecting the LLMs for this work, we considered criteria such as availability, popularity, and scalability. First, in terms of availability, we had access to one cloud provider during the execution of this study, namely OpenAI. As such, we selected the flagship GPT4o available for this provider at the time of the study. For other models, since one of our objectives is to discuss the advantages of different ways of deploying these models, we opted to utilize the Ollama[4] tool to deploy Llama 3.1 (from Meta), and Mistral and Gemma (from Google). In terms of popularity, the selected open-source models belonged to highly adopted model families on Hugging Face at the time of model selection, consistently appearing among the most downloaded text-generation models on the platform[5]. Popularity was assessed using the top 20 download-based rankings available during the study period. Finally, regarding scalability, all models chosen are available in different variations, which mainly affect parameter count. The parameter count of a model dramatically affects its performance, especially inference speed and memory requirements. A higher parameter count can also be associated with higher reasoning capabilities. The names and characteristics of all models chosen are available in Table 1.

As a comparison baseline, we have also evaluated four widely used pre-trained transformer architectures for classification: BERT, XLNet, RoBERTa, and ALBERT. Concretely, we used the Hugging Face models `bert-base-cased`, `xlnet-base-cased`, `roberta-base`, and `albert-base-v1`. We chose this model set as it mirrors the set utilized by Zhang et al. (2020), as we intend to reproduce its experimental setup for comparison purposes.

[4] https://ollama.com/

[5] https://huggingface.co/models?pipeline_tag=text-generation&sort=downloads

3) Prompt Engineering. In terms of prompt engineering, we started with a base prompt which can be seen in Listing 1, utilizing the role prompting technique, in which you provide a specific role for the model to perform. This prompt follows the same emotion-to-polarity rationale used in prior datasets (Novielli et al. 2020) grounded in Shaver et al. (1987).

```
You are a bot that classifies messages from Github pull
requests.
Classify the message as one of three types of sentiment:
positive, neutral, and negative.

For classification purposes, we consider love and joy
(and related emotions) positive,
anger, sadness, and fear, negative,
surprise can be positive or negative depending on the
context,
and neutral is considered the absence of any emotions.

Return the result one of the following JSONs:
{{"sentiment": "positive"}}, {{"sentiment": "negative"}}
OR {{"sentiment": "neutral"}}.

Here are some examples to help you get started:
{examples}

Message: "{message}"
```

Listing 1 Prompt template utilized for classification

That prompt was utilized for RQ_1 (except for the line that mentions examples). For RQ_2 , we selected a number of additional prompt engineering techniques that have been used in previous works to successfully improve the results of classification tasks (Marvin et al. 2023). As such, the prompt engineering techniques chosen are related to how we can provide examples to the LLM. We selected three techniques, which are: (i) *one-shot*, providing one example to the LLM; (ii) *few-shot*, providing three or more examples to the LLM; and (iii) *chain of thought*, providing one or more examples to the LLM, which contain the reasoning behind each response (one of these examples can be seen in Listing 2). The reasoning is only added in Chain of Thought prompts.

We note that the examples used in the one-shot, few-shot, and chain-of-thought prompting strategies were not explicitly excluded from the evaluation/test corpus. In our study, the prompt examples were not used to train or fine-tune the models, but rather served as contextual instructions provided at inference time. Consequently, the notion of separating training and evaluation data differs from that in traditional supervised machine learning settings, where evaluation sets are used to assess performance during model training or evaluate models that are trained directly on labeled data. Additionally, not all prompting configurations included examples, and those that did relied on a limited number of illustrative cases. While overlapping examples may introduce contextual bias, we expect its impact to be limited in this setting and acknowledge this design choice as a methodological consideration when interpreting the results.

We experimented with alternative formatting structures during our pilot phase, but opted to present the examples in a single structured block within the prompt to maintain context consistency and ensure strictly valid JSON outputs. Regarding the selection process, the examples utilized in the prompts were manually selected from the dataset by two authors

through a paired review process. Instead of random sampling, which might introduce ambiguous or low-quality examples, we prioritized clear, unambiguous instances for each sentiment class. In summary, the authors looked for examples where both authors could agree that they were only able to identify one type of polarity in a message (as some messages can express multiple polarities at varying intensities). This ensured that the models received clear instructions on the definitions of positive, negative, and neutral sentiments within this specific domain.

```
Example 1: Oh, you know what? I'm stupid.
I'm actually using 'http-server'. Please, accept my apologies.
Response for Example 1: {{"sentiment": "negative"}}
Reasoning for Example 1: In this message, the author utilizes
a self deprecating expression (I'm stupid) and then apologizes,
likely to the reviewer.
```

4) Model Execution. As aforementioned, we mainly utilized two providers: OpenAI (cloud-based) and Ollama (local) to execute the chosen LLMs. The locally-deployed LLMs were distributed in a computer with the following specifications: a desktop with an AMD Ryzen 7 9800X3D, 64GB of RAM, and an NVIDIA RTX 5080 GPU with 16GB of VRAM. We configured the models to be as deterministic as possible by setting the *temperature* parameter to 0, prioritizing reproducibility across runs.

To validate this approach, we conducted a consistency test across all ten models using a sample of 50 messages over three independent runs. We calculated the Cohen's κ inter-rater reliability metric to determine if there were significant agreement between the runs. As summarized in Table 2, nine models achieved perfect agreement ($\kappa = 1.0$). While we acknowledge that intrinsic randomness can arise from variations in hardware architectures or software implementation details, our results confirm that the randomness is negligible on fixed hardware. Consequently, a single-run design was deemed appropriate for this study.

We note that setting the temperature to 0 introduces a trade-off between determinism and potential classification performance. Lower temperature values promote more deterministic outputs, improving reproducibility across runs. This is especially relevant for this study, where more deterministic output can facilitate reliable comparisons across models and configurations. However, higher temperature values may enable models to explore alternative token probabilities, which could potentially improve predictive performance in some scenarios. In this study, we prioritized deterministic behavior to ensure reproducibility and comparability of results across multiple models. We acknowledge,

Table 2 Model consistency and reproducibility metrics (3 runs, $n = 50$, temperature $= 0$)

Model	Provider	Avg. κ	Std. Dev.
mistral-nemo:12b	Ollama	1.000	0.000
mistral-small:22b	Ollama	1.000	0.000
gemma2:9b	Ollama	1.000	0.000
gemma2:27b	Ollama	1.000	0.000
llama3.1:8b	Ollama	1.000	0.000
llama3.1:70b	Ollama	1.000	0.000
deepseek-r1:8b	Ollama	1.000	0.000
deepseek-r1:32b	Ollama	1.000	0.000
gpt-4o-mini-2024-07-18	OpenAI	1.000	0.000
gpt-4o-2024-05-13	OpenAI	0.911	0.016

however, that this choice may limit the exploration of configurations that could yield improved average performance, and we did not explicitly evaluate the impact of varying temperature settings on model performance. Investigating this trade-off remains an opportunity for future work.

Finally, regarding the the transformer models (i.e., BERT, RoBERTa, XLNet, and ALBERT), we implemented a fine-tuning and evaluation pipeline based on an adapted version of the scripts made available by Zhang et al. (2020). We first load both datasets, which are then tokenized using the corresponding model tokenizer and encoded into fixed-length sequences with attention masks, and sentiment labels are mapped to numeric IDs for training. Each model is then fine-tuned using an 80/20 train–test split (stratified by label), using the same set of parameters as Zhang et al. (2020) (i.e., using batches of size 16, 4 epochs for fine tuning, and parameter updates are computed using the AdamW optimizer with a learning rate of 2e-5). Both training and inference were executed using the same hardware setup as the LLMs.

5) Result Analysis. We utilized Python scripts (available as part of the replication package Coutinho et al. 2025a) that leveraged the pandas library[6] to analyze the output of the LLMs in terms of precision, recall, and F1-score. To provide a comprehensive view of performance, especially in a multi-class scenario like ours, we also report the macro and micro averages of these metrics. The **macro average** computes the metric independently for each class and then takes the unweighted average, treating all classes as equally important regardless of their size. In contrast, the **micro average** aggregates the contributions of all classes to compute the average metric, which means it is weighted by the number of instances in each class. Since, depending on the goal of the user when utilizing sentiment analysis, either metric can be relevant, we discuss the results in terms of different possible usage scenarios (see Section 5.2).

This evaluation strategy is consistent with prior work on sentiment analysis in SE. Studies (Lin et al. 2018; Novielli et al. 2020) report macro-averaged F1-scores as a primary metric, while also analyzing class-level precision and recall, particularly for negative sentiment. More recent work (Obaidi et al. 2022, 2025b, a) further demonstrates that performance rankings based on aggregated metrics can vary substantially across GitHub-based datasets, reinforcing the importance of reporting both aggregated and class-specific metrics when evaluating sentiment analysis tools in software engineering contexts.

Additionally, to better understand our results, we performed a qualitative evaluation of our misclassified instances. For this evaluation, we randomly sampled 30% of the cases (114 messages) in which our best-performing model configuration got the classification wrong (according to the labeled dataset) and performed an open coding process in which two researchers labeled the characteristics of the message that could possibly have affected the results.

The open coding process was conducted over two main iterations. In the first iteration, the two researchers independently coded the sampled misclassified cases and iteratively refined an initial set of codes as new characteristics emerged from the data. In the second iteration, code saturation was reached, as no substantially new categories were required to describe additional cases. At this stage, previously analyzed messages were revisited and re-coded where necessary to ensure consistency with the final version of the codebook.

[6] https://pandas.pydata.org/

The coding process was structured as follows:

1. Coding: Coders assigned one or more codes from the codebook to describe the key characteristics of the message.
2. Codebook Expansion: When a coder identified a new characteristic not yet covered by existing codes, a new label was proposed and added to the codebook, along with a concise definition. For example, up to message #7, no code captured the phenomenon of emotionally charged expressions appearing at the beginning of a message. Recognizing this gap, one of the coders introduced the category *Sentiment Cue at Start*, which, after review and approval by the research team, was added to the codebook. Subsequently, this category emerged as one of the codes most frequently applied in the analysis.
3. Conflict Resolution: In cases where coders disagreed on the appropriate codes for a given message, a third researcher reviewed the message and adjudicated to reach consensus.

5 Results and Discussion

The following subsections present the results for each RQ. Based on the results, we highlight the most important findings.

5.1 Baseline Results (Transformer-based models)

This section reports the performance of pre-trained transformer-based sentiment analysis models when applied to GitHub pull request discussions, using both the Gold dataset and the PRemo dataset.

Table 3 presents the results obtained on the Gold dataset. Overall, all evaluated transformer-based models achieved consistently high performance across sentiment classes, with macro-averaged F1-scores ranging between 91% and 92%. RoBERTa achieved the strongest overall results, reaching a macro-averaged F1-score of 92% and demonstrating balanced precision and recall across all three sentiment classes. In particular, it obtained the highest F1-scores for the negative and neutral classes (91% and 92%, respectively). XLNet and ALBERT followed closely, both achieving macro-averaged F1-scores of 91%, while BERT exhibited comparable performance with slightly higher precision for the positive class (94%). These results indicate that, when evaluated on a controlled and relatively clean dataset, fine-tuned transformer-based models are highly effective at sentiment classification.

Table 4 reports the performance of the same models on the PRemo dataset. In contrast to the Gold dataset, a substantial performance degradation is observed across all models. Macro-averaged F1-scores drop to the 67%-72% range, highlighting the increased difficulty of sentiment analysis in authentic software engineering communication. Among the evaluated models, XLNet achieved the highest macro-averaged F1-score (72%), followed closely by RoBERTa and BERT (both at 71%). ALBERT showed the weakest overall performance, particularly for the negative class, where recall dropped to 40%, resulting in an F1-score of only 52%.

A consistent pattern across all models is the difficulty in detecting negative sentiment. While precision for negative sentiment remains moderate, recall is substantially lower than for positive and neutral classes. This suggests that many negative messages are misclassi-

Table 3 Performance of transformer-based models on the Gold dataset

Model	Class	Precision	Recall	F1-score
ALBERT	Positive	92%	92%	92%
	Negative	88%	**92%**	90%
	Neutral	**92%**	90%	91%
	Macro Avg.	91%	91%	91%
	Micro Avg.	91%	91%	91%
BERT	Positive	**94%**	91%	**93%**
	Negative	90%	88%	89%
	Neutral	89%	**92%**	91%
	Macro Avg.	91%	91%	91%
	Micro Avg.	91%	91%	91%
ROBERTA	Positive	90%	95%	92%
	Negative	**93%**	90%	**91%**
	Neutral	92%	91%	**92%**
	Macro Avg.	**92%**	**92%**	**92%**
	Micro Avg.	**92%**	**92%**	**92%**
XLNET	Positive	90%	**95%**	92%
	Negative	92%	89%	90%
	Neutral	92%	91%	91%
	Macro Avg.	91%	92%	91%
	Micro Avg.	91%	91%	91%

Table 4 Performance of transformer-based models on the PRemo dataset

Model	Class	Precision	Recall	F1-score
ALBERT	Positive	75%	75%	75%
	Negative	**74%**	40%	52%
	Neutral	67%	**83%**	74%
	Macro Avg.	72%	66%	67%
	Micro Avg.	70%	70%	70%
BERT	Positive	**81%**	75%	78%
	Negative	72%	51%	60%
	Neutral	69%	82%	**75%**
	Macro Avg.	**74%**	69%	71%
	Micro Avg.	**72%**	**72%**	**72%**
ROBERTA	Positive	77%	78%	78%
	Negative	58%	**80%**	**67%**
	Neutral	**78%**	62%	69%
	Macro Avg.	71%	**73%**	71%
	Micro Avg.	71%	71%	71%
XLNET	Positive	80%	**82%**	**81%**
	Negative	57%	66%	61%
	Neutral	77%	69%	73%
	Macro Avg.	71%	73%	**72%**
	Micro Avg.	72%	72%	72%

fied as neutral, likely due to the prevalence of technical criticism expressed in a factual or emotionally subdued tone. Neutral sentiment, by contrast, tends to achieve higher recall but lower precision, indicating a tendency for models to over-predict neutrality in ambiguous cases. These results reinforce prior findings that sentiment analysis in software engineering contexts is significantly more challenging than in general-domain text, and that models fine-tuned on conventional sentiment corpora struggle to generalise to developer discussions without domain-specific adaptation.

Part of the observed performance gap between the Gold and PRemo datasets can be attributed to structural differences between the two corpora. The Gold dataset provides a larger volume of labeled training data, which likely facilitates more stable fine-tuning and better parameter estimation for transformer-based models. In contrast, messages in PRemo are longer on average. These characteristics increase contextual ambiguity and may dilute sentiment cues, particularly for negative sentiment, which is often expressed indirectly. As a result, even strong transformer-based baselines exhibit reduced recall when applied to PRemo (see Section 5.4 for more details, as some of these points were also observed as challenging factors for LLMs on our qualitative evaluation).

Finding 1: Transformer-based models achieve near-ceiling performance on the Gold dataset, with macro-averaged F1-scores ranging from 91% to 92%, but their performance drops substantially on the PRemo dataset, where macro F1-scores fall to the 67%-72% range, largely due to reduced recall for negative sentiment in real-world PR discussions.

5.2 Performance of Large Language Models in a Zero-shot Setup (RQ_1)

This section evaluates the effectiveness of ten large language models (LLMs) for sentiment analysis in PR discussions. Tables 5 and 6 summarize the performance metrics (Precision, Recall, and F1-score) for all evaluated models on the Gold and PRemo datasets, respectively, covering both proprietary and open-source options. We highlighted in bold the best model results for each class and each metric. The reported sample size (n) may vary slightly between models, as responses that did not conform to the expected output format were considered hallucinations and therefore excluded from our evaluation. We analyze these results with a focus on how model behavior changes across datasets, sentiment classes, and deployment constraints.

Across both datasets, proprietary models from the GPT family exhibit strong but uneven performance. On the Gold dataset, GPT-4o achieves high precision across all sentiment classes, with values reaching up to 96% for positive sentiment, but shows comparatively lower recall for positive and negative classes (62% and 82%, respectively). This indicates conservative classification behavior, favoring correctness over coverage even in a controlled evaluation setting. On the PRemo dataset, this pattern becomes more pronounced. GPT-4o maintains high precision for positive (92%) and negative (77%) sentiment, but recall drops substantially for both classes (45% and 40%). At the same time, recall for neutral sentiment increases to 93%, accompanied by low precision (59%), suggesting a strong bias towards neutral predictions when faced with ambiguous, real-world PR discussions.

Compared to the previously mentioned fine-tuned transformer baselines, which achieve near-ceiling macro F1-scores on Gold and substantially higher recall for negative sentiment

Table 5 Zero-shot performance for all models tested, for the Gold dataset

Model	Class	Precision	Recall	F1-score
deepseek-r1-32b (n=7122)	Positive	92%	79%	85%
	Negative	**82%**	75%	79%
	Neutral	76%	**87%**	**81%**
	Macro Avg.	**83%**	**80%**	**81%**
	Micro Avg.	**81%**	**81%**	**81%**
deepseek-r1-8b (n=7122)	Positive	77%	85%	81%
	Negative	72%	79%	75%
	Neutral	80%	69%	74%
	Macro Avg.	76%	78%	77%
	Micro Avg.	76%	76%	76%
gemma2-27b (n=7122)	Positive	81%	**91%**	86%
	Negative	66%	**85%**	74%
	Neutral	**86%**	62%	72%
	Macro Avg.	78%	79%	77%
	Micro Avg.	77%	77%	77%
gemma2-9b (n=7122)	Positive	88%	80%	84%
	Negative	67%	77%	72%
	Neutral	74%	71%	73%
	Macro Avg.	77%	76%	76%
	Micro Avg.	76%	76%	76%
gpt-4o (n=7122)	Positive	96%	62%	75%
	Negative	76%	82%	**79%**
	Neutral	72%	85%	78%
	Macro Avg.	81%	76%	77%
	Micro Avg.	78%	78%	78%
gpt-4o-mini (n=7122)	Positive	93%	76%	84%
	Negative	79%	74%	77%
	Neutral	73%	85%	79%
	Macro Avg.	82%	79%	80%
	Micro Avg.	80%	80%	80%
llama3.1-70b (n=7112)	Positive	88%	85%	**86%**
	Negative	70%	81%	75%
	Neutral	81%	73%	77%
	Macro Avg.	79%	80%	79%
	Micro Avg.	79%	79%	79%
llama3.1-8b (n=7096)	Positive	77%	87%	82%
	Negative	59%	84%	70%
	Neutral	82%	50%	62%
	Macro Avg.	73%	74%	71%
	Micro Avg.	71%	71%	71%
mistral-nemo-12b (n=7068)	Positive	91%	76%	83%
	Negative	67%	81%	73%
	Neutral	76%	72%	74%
	Macro Avg.	78%	76%	77%
	Micro Avg.	76%	76%	76%
mistral-small-22b (n=7119)	Positive	**96%**	73%	83%
	Negative	75%	81%	78%
	Neutral	74%	82%	78%

Table 5 (continued)

Model	Class	Precision	Recall	F1-score
	Macro Avg.	82%	78%	79%
	Micro Avg.	79%	79%	79%

on PRemo, GPT-4o's zero-shot behavior reveals a clear trade-off: strong precision but markedly reduced coverage for polarized sentiment.

> **Finding 2:** Proprietary models such as GPT-4o achieve high precision across evaluation settings (up to 96%) but show a systematic recall degradation for positive and negative sentiment when faced with noisier, real-world PR discussions, with recall values falling to the 40%–46% range.

GPT-4o mini presents a different trade-off. On both datasets, it consistently improves recall for the positive class compared to GPT-4o, increasing from 62% to 76% on Gold and from 45% to 65% on PRemo. However, these gains are offset by reduced recall for negative sentiment, particularly on PRemo, where recall drops to 28%. This highlights that improvements in one sentiment class often come at the expense of others, even within the same model family.

It is important to note that while we report Macro and Micro F1-scores to allow for high-level comparisons, our discussion primarily focuses on granular Precision and Recall metrics. In the context of software engineering tools, performance requirements are rarely symmetric. For instance, toxicity detection demands high precision to avoid false alarms, while feedback analysis requires high recall to capture all relevant inputs. Relying on aggregated F1-scores would obscure these specific capabilities and trade-offs. This makes it difficult to recommend models for distinct practical applications.

> **Finding 3:** Proprietary models like GPT-4o show high precision but struggle with recall. Smaller counterparts like GPT-4o mini can improve performance on specific metrics (e.g., positive recall) but often at the cost of others, highlighting performance trade-offs even within the same model family.

When discussing the remaining models, a key factor is the trade-off between proprietary, cloud-based models and open-source, locally-deployable alternatives. When utilizing proprietary models, such as the GPT family, users offload processing and maintenance to a cloud provider. While this simplifies deployment, it introduces concerns regarding data privacy, dependency on provider uptime, and accumulating operational costs. In our experiments, the cost disparity between proprietary tiers was significant: while GPT-4o-mini cost only \$0.28 USD for approximately 28,800 requests, the flagship GPT-4o model accounted for \$50.66 USD for a similar volume.

In contrast, open-source LLMs offer greater control, as while they can be cloud-hosted, they can also be hosted locally. This provides enhanced privacy (as data can remain local)

Table 6 Zero-shot performance for all models tested, for the PRemo dataset

Model	Class	Precision	Recall	F1-score
deepseek-r1-32b (n=1752)	Positive	86%	66%	74%
	Negative	71%	39%	51%
	Neutral	63%	84%	72%
	Macro Avg.	73%	63%	66%
	Micro Avg.	**69%**	68%	69%
deepseek-r1-8b (n=1486)	Positive	84%	34%	49%
	Negative	63%	26%	36%
	Neutral	54%	70%	61%
	Macro Avg.	67%	43%	49%
	Micro Avg.	59%	49%	54%
gemma2-27b (n=1790)	Positive	63%	82%	71%
	Negative	68%	51%	**58%**
	Neutral	69%	65%	67%
	Macro Avg.	67%	66%	66%
	Micro Avg.	67%	67%	67%
gemma2-9b (n=1790)	Positive	77%	68%	72%
	Negative	67%	43%	53%
	Neutral	63%	79%	70%
	Macro Avg.	69%	64%	65%
	Micro Avg.	67%	67%	67%
gpt-4o (n=1791)	Positive	**92%**	45%	61%
	Negative	**77%**	40%	52%
	Neutral	59%	**93%**	72%
	Macro Avg.	**76%**	59%	62%
	Micro Avg.	66%	66%	66%
gpt-4o-mini (n=1791)	Positive	81%	65%	72%
	Negative	77%	28%	41%
	Neutral	61%	89%	**72%**
	Macro Avg.	73%	61%	62%
	Micro Avg.	67%	67%	67%
llama3.1-70b (n=1791)	Positive	73%	77%	**75%**
	Negative	69%	40%	51%
	Neutral	65%	77%	71%
	Macro Avg.	69%	65%	66%
	Micro Avg.	68%	68%	68%
llama3.1-8b (n=1791)	Positive	55%	**85%**	67%
	Negative	60%	**56%**	58%
	Neutral	**70%**	49%	58%
	Macro Avg.	62%	63%	61%
	Micro Avg.	61%	61%	61%
mistral-nemo-12b (n=1783)	Positive	76%	73%	75%
	Negative	66%	50%	57%
	Neutral	66%	76%	71%
	Macro Avg.	70%	**66%**	**67%**
	Micro Avg.	69%	**69%**	**69%**
mistral-small-22b (n=1790)	Positive	92%	50%	65%
	Negative	71%	41%	52%
	Neutral	60%	89%	72%

Table 6 (continued)

Model	Class	Precision	Recall	F1-score
	Macro Avg.	74%	60%	63%
	Micro Avg.	66%	66%	66%

and zero marginal inference cost after an initial hardware investment. In this work, our local deployment utilized a setup costing approximately \$3,500 USD (costs may vary depending on region). Depending on the model and the scale needed, this initial investment may not be preferable. Even in our setup, the quantized version of Llama3.1 70B required approximately 40GB of system memory in our setup, thus some of the model had to run on the CPU, which made inference considerably slower. While local hosting entails ongoing electricity and maintenance, it can avoid the scaling costs of proprietary APIs. For workloads similar to the ones tested in this paper (i.e., similar prompt size and output size), processing one million messages with GPT-4o-mini would cost approximately \$9.72 USD, whereas GPT-4o would scale to roughly \$1,759 USD. This flexibility and long-term cost-predictability make open-source models ideal for building sustainable open-source tools.

Open-source models present a different set of trade-offs. On the Gold dataset, several open-source models achieve performance comparable to proprietary alternatives, with macro-averaged F1-scores clustering in the 71%-81% range. Models such as Llama3.1 70B, Mistral Small, and Deepseek-R1 32B demonstrate balanced precision and recall across sentiment classes, indicating that large-scale open models can effectively capture sentiment in structured evaluation settings. However, even the strongest open-source LLMs remain well below the supervised transformer baselines on the Gold dataset (91–92% macro F1), highlighting the performance gap between zero-shot inference and fine-tuned models when high-quality labeled data is available.

On the PRemo dataset, overall performance decreases for all models, but open-source LLMs remain competitive, with macro-averaged F1-scores ranging from 49% to 67%. While some models such as Mistral Nemo and Gemma2 27B achieve comparatively higher macro F1-scores (67% and 66% respectively), their performance remains markedly asymmetric across sentiment classes, particularly due to reduced recall for negative sentiment (50% and 51% respectively) and a tendency to over-predict neutral labels.

Reliability also varies across models. Some configurations, particularly Deepseek-R1 8B, produced a non-negligible number of hallucinations, failing to generate valid outputs for $\approx 17\%$ of messages. In contrast, models such as Mistral Small classified nearly all instances successfully, combining stable output formatting with competitive performance. Cases in which the model was not able to provide its output in a readable format were excluded from the calculation of the metric. The number of considered messages can be observed by looking at the N value besides the model name in Tables 5 and 6.

Finding 4: Open-source LLMs achieve performance comparable to proprietary models on both datasets, with macro F1-scores up to 81% on Gold and up to 67% on PRemo, while offering greater control over deployment, cost, and data privacy.

A universal trend observed is the superior performance of all evaluated LLMs over traditional, non-LLM-based sentiment analysis tools, especially in key places such as the negative class. In a previous study by the authors of this work, several traditional tools were evaluated using the same dataset (Coutinho et al. 2024). As shown in Table 7, the traditional tool that performs best against negative sentiment (Senti4SD) achieved a precision of only 57%. In contrast, every LLM in our study surpassed this, with the lowest precision for the negative class being 60% (Llama3.1 8B). All traditional tools were used in their original, pre-trained form, as released by their authors, without any additional training or fine-tuning, mirroring the experimental setup adopted for the LLMs. This demonstrates a significant leap in performance, establishing LLMs as a more effective technology for this task.

We note that the results presented in Table 7 were reproduced directly from our previous work (Coutinho et al. 2024) without modification. As such, we intentionally preserved the original reported metrics to ensure fidelity to the prior evaluation and to avoid introducing inconsistencies that could arise from recomputing or partially reconstructing results. In particular, micro-average values were not reported in the original study. Therefore, we retained the original metrics as published and acknowledge this limitation when interpreting comparisons with the LLM-based approaches.

Finding 5: Open-source models such as Llama3.1 and Mistral Small are highly competitive, with some outperforming larger models on specific metrics and offering a better balance of performance, reliability, and resource efficiency. All evaluated LLMs, regardless of size or type, show improved performance compared to traditional sentiment analysis tools from the literature on the same dataset.

The practical implications of these metrics are highly dependent on the specific application. As highlighted in our findings, evaluating a model's performance requires considering the task for which it will be employed. We present some examples in the following:

Table 7 Performance metrics for non-LLM tools available in the literature. Reported by Coutinho et al. (2024) utilizing the PRemo dataset

Class	Tool	Precision	Recall	F1-score
Positive	SentiStrength	55%	**79%**	65%
	SentiStrengthSE	**79%**	63%	**70%**
	DEVA	66%	72%	69%
	Senti4SD	56%	71%	63%
Negative	SentiStrength	45%	70%	55%
	SentiStrengthSE	43%	**80%**	56%
	DEVA	48%	70%	**57%**
	Senti4SD	**57%**	42%	49%
	SentiCR	43%	40%	41%
Neutral	SentiStrength	**80%**	36%	50%
	SentiStrengthSE	79%	55%	65%
	DEVA	80%	57%	**66%**
	Senti4SD	66%	**64%**	65%
Macro Avg.	SentiStrength	60%	62%	61%
	SentiStrengthSE	**67%**	66%	**67%**
	DEVA	60%	59%	60%
	Senti4SD	65%	**66%**	66%

- **Toxicity Detection**: In a scenario focused on toxicity detection, the goal would be to automatically identify and flag for moderation comments that are toxic, for example, ones that are either: rude, disrespectful, or abusive. To minimize false positives (i.e., incorrectly flagging non-toxic comments), high precision in the negative class would be crucial. This would ensure that comments are only flagged when confidence is high, preventing the mislabeling of constructive, yet critical, feedback which could stifle important project discussions. For such a task, models like GPT-4o (78%), GPT-4o mini (77%), and Mistral Small (72%) would be strong candidates.
- **Identifying Informative Feedback**: If the use case were to gather all constructive feedback to guide project improvement, it would involve identifying comments that contain praise, suggestions, or feature requests. Importantly, such feedback is not inherently positive in sentiment: informative and actionable comments may be neutral or even negative in tone while remaining constructive and valuable to the development process (Hata et al. 2022). In this context, sentiment analysis should not be interpreted as a direct proxy for informativeness. Instead, sentiment signals can support the identification of candidate messages for further analysis, particularly when combined with additional heuristics or manual inspection. For scenarios where avoiding missed constructive input is critical, higher recall in the positive class would be essential to avoid overlooking valuable input from the community. In such a scenario, models like Llama3.1 8B (85%) and GPT-4o mini (66%) would be suitable, outperforming GPT-4o's low recall.
- **Automated Support Team Assignment**: For a project with dedicated support, a hypothetical task could involve automatically triaging user messages. To ensure all user issues are addressed, high recall in the negative class would be critical to capture every dissatisfied or problematic comment and route it to the appropriate team for intervention. Llama3.1 8B would show the highest recall here (56%), though this would come at the cost of lower precision.
- **General-Purpose Analysis**: For a task such as creating a dashboard to track overall sentiment trends over time, a balanced performance across all classes would be ideal. In this context, the goal would be to get a reliable overview of the community's mood, where no single sentiment class is inherently more important than another. Therefore, a model's average F1-score, which reflects a balance between precision and recall across all classes, would be the most suitable metric. For this purpose, Mistral Nemo (68% macro F1) and GPT-4o (62% macro F1) would show the most balanced performance.

Finding 6: No single LLM excels universally. The optimal choice is highly dependent on the specific task and its corresponding metric priorities, which must be balanced against practical constraints such as cost, hardware availability, and data privacy requirements.

5.3 Performance Using Prompt Engineering Techniques (RQ_2)

5.3.1 One-shot Technique

This technique consists of providing a single example to the LLM as part of the prompt. As it requires only a single example, it minimizes both the effort of selecting examples

and the size of the input that is passed to the model. In our case, as sentiment analysis is a multi-class classification task, we provided three examples, one for each class (i.e., one-shot per class) . Some of our pilot experiments using a true single-example prompt showed that providing a single example from only one class overall made performance metrics worse for the other classes, as the models tended to be biased toward predicting the class of the provided example. This was still observed even in more complex techniques such as few-shot and CoT, so we opted to balance the examples per class. Tables 8 and 9 report the results obtained with one-shot prompting on the Gold and PRemo datasets, respectively, along with the relative change compared to the zero-shot baseline. The prompt examples are provided in the supplementary material (Coutinho et al. 2025a).

On the Gold dataset, one-shot prompting generally led to modest changes in overall performance, with macro-averaged F1-scores remaining in a narrow range across most models. Improvements were primarily observed in recall for the neutral class, often accompanied by corresponding drops in precision. For instance, Gemma2 27B and Llama3.1 8B increased neutral recall by over 20%, while precision decreased by 10% and 11% respectively. These shifts suggest that, in a controlled dataset, the addition of a single example helps models expand decision boundaries without substantially improving class discrimination.

For positive sentiment on the Gold dataset, some models benefited from one-shot prompting through increased recall, while others exhibited reduced precision. Mistral Small achieved the highest precision (96%) but at the cost of recall (62%), whereas GPT-4o mini achieved the highest recall (also 96%) with a precision drop (66%). Despite these opposing behaviours, the best F1-scores for the positive class (85%) were achieved by Gemma2 27B, reflecting different precision–recall trade-offs rather than uniformly improved performance.

On the PRemo dataset, for the Positive class, Mistral Small achieved the highest precision (85%), showing strong capability in correctly identifying positive instances, but at the expense of recall (62%). In contrast, Llama3.1 8B exhibited the highest recall (93%) but suffered a notable drop in precision (47%), indicating a tendency to over-predict positives. The best overall F1-score (79%) was jointly obtained by GPT-4o and Mistral nemo, demonstrating balanced precision–recall trade-offs. Models like Gemma2 27B and GPT-4o mini performed consistently but without reaching top precision or recall.

For the Negative class, GPT-4o and Mistral nemo achieved the highest recall (62%), showing better sensitivity to negative sentiment detection. Llama3.1 70B recorded the highest precision (76%) but with a recall drop to 52%, indicating a preference for precision. F1-scores were modest overall, with Mistral nemo leading at 67%. Lower-tier performances, such as Deepseek-R1 8B (30%), revealed challenges in capturing negative sentiment.

For the Neutral class, performance was more evenly distributed. GPT-4o and Mistral nemo both achieved the highest F1-score (76%), with Mistral small excelling in recall (85%) and GPT-4o leading in precision (76%). Smaller models, such as Llama3.1 8B, exhibited dramatic recall drops (36%) despite moderate precision (73%).

When comparing the one-shot performance to the zero-shot baseline, a general improvement in macro-level F1-scores was observed for most large models. In particular, GPT-4o and Mistral Nemo demonstrated the most consistent improvements, achieving balanced precision and recall across all sentiment classes in the one-shot setting, whereas in zero-shot they showed a tendency toward heterogeneous values. The provision of an example appears to have reduced overgeneralization, leading to more targeted predictions, especially for the Positive class.

Table 8 Performance with balanced one-shot prompting for the gold dataset (improvement vs baseline shown in parentheses)

Model	Class	Precision	Recall	F1-score
deepseek-r1-32b (n=7121)	Positive	94% (+2%)	73% (-6%)	82% (-3%)
	Negative	82% (0%)	74% (-1%)	**78%** (-1%)
	Neutral	73% (-3%)	89% (+2%)	**80%** (-1%)
	Macro Avg.	83% (0%)	79% (-2%)	80% (-1%)
	Micro Avg.	80% (-1%)	80% (-1%)	80% (-1%)
deepseek-r1-8b (n=7122)	Positive	89% (+13%)	75% (-10%)	81% (+1%)
	Negative	74% (+1%)	**77%** (-2%)	75% (0%)
	Neutral	75% (-5%)	81% (+11%)	77% (+3%)
	Macro Avg.	79% (+3%)	77% (0%)	78% (+1%)
	Micro Avg.	78% (+1%)	78% (+1%)	78% (+1%)
gemma2-27b (n=7122)	Positive	89% (+8%)	82% (-9%)	**85%** (0%)
	Negative	79% (+13%)	72% (-13%)	76% (+1%)
	Neutral	**76%** (-10%)	85% (+22%)	80% (+8%)
	Macro Avg.	81% (+4%)	**80%** (0%)	**80%** (+3%)
	Micro Avg.	**80%** (+3%)	**80%** (+3%)	**80%** (+3%)
gemma2-9b (n=7122)	Positive	89% (+1%)	78% (-2%)	83% (-1%)
	Negative	71% (+4%)	72% (-5%)	72% (0%)
	Neutral	72% (-2%)	77% (+6%)	75% (+2%)
	Macro Avg.	77% (+1%)	76% (0%)	76% (0%)
	Micro Avg.	76% (+1%)	76% (+1%)	76% (+1%)
gpt-4o (n=7122)	Positive	95% (-1%)	64% (+2%)	76% (+1%)
	Negative	86% (+9%)	70% (-12%)	77% (-2%)
	Neutral	69% (-2%)	93% (+8%)	79% (+2%)
	Macro Avg.	83% (+2%)	76% (-1%)	78% (0%)
	Micro Avg.	78% (0%)	78% (0%)	78% (0%)
gpt-4o-mini (n=7122)	Positive	95% (+1%)	70% (-6%)	81% (-4%)
	Negative	**92%** (+13%)	55% (-20%)	69% (-8%)
	Neutral	66% (-8%)	**96%** (+10%)	78% (-1%)
	Macro Avg.	**84%** (+2%)	73% (-5%)	76% (-4%)
	Micro Avg.	76% (-3%)	76% (-3%)	76% (-3%)
llama3.1-70b (n=7121)	Positive	90% (+2%)	78% (-7%)	84% (-3%)
	Negative	86% (+16%)	49% (-32%)	63% (-12%)
	Neutral	67% (-14%)	92% (+19%)	77% (+1%)
	Macro Avg.	81% (+1%)	73% (-7%)	75% (-5%)
	Micro Avg.	76% (-3%)	76% (-3%)	76% (-3%)
llama3.1-8b (n=7114)	Positive	78% (+1%)	**83%** (-4%)	80% (-1%)
	Negative	70% (+11%)	63% (-21%)	67% (-3%)
	Neutral	71% (-11%)	73% (+23%)	72% (+10%)
	Macro Avg.	73% (0%)	73% (-1%)	73% (+2%)
	Micro Avg.	73% (+2%)	73% (+2%)	73% (+2%)
mistral-nemo-12b (n=7059)	Positive	93% (+2%)	74% (-1%)	83% (0%)
	Negative	83% (+16%)	66% (-15%)	73% (0%)
	Neutral	71% (-5%)	89% (+17%)	79% (+5%)
	Macro Avg.	82% (+4%)	76% (0%)	78% (+2%)
	Micro Avg.	79% (+2%)	78% (+2%)	78% (+2%)
mistral-small-22b (n=7119)	Positive	**96%** (0%)	62% (-10%)	76% (-7%)
	Negative	85% (+9%)	60% (-21%)	70% (-8%)

Table 8 (continued)

Model	Class	Precision	Recall	F1-score
	Neutral	64% (-9%)	92% (+10%)	76% (-2%)
	Macro Avg.	82% (0%)	72% (-7%)	74% (-6%)
	Micro Avg.	74% (-5%)	74% (-5%)	74% (-5%)

The most notable relative improvement occurred in the Negative class, where several models — including Mistral Small and Gemma2 27B — displayed higher F1-scores compared to zero-shot. This aligns with prior evidence that negative sentiment is harder to detect without explicit cues, and the single in-context example likely provided clearer polarity boundaries. However, some models, such as Deepseek-R1 8B, saw limited or even negative changes in performance, which may be due to sensitivity to prompt bias. For instance, the single example could unintentionally bias the model toward borderline interpretations.

Across both datasets, one-shot prompting did not consistently lead to higher macro-averaged F1-scores. Instead, its primary effect was a redistribution of errors across sentiment classes, typically increasing recall for harder classes such as negative sentiment while reducing precision or shifting predictions towards neutrality. This pattern was more evident on the PRemo dataset, where linguistic ambiguity and technical context amplify the influence of in-context examples.

Finding 7: One-shot prompting primarily reshapes precision–recall trade-offs rather than uniformly improving performance, with modest effects on the Gold dataset and larger, but more asymmetric, shifts on the PRemo dataset, particularly for negative sentiment.

5.3.2 Few-shot Technique

This technique consists of providing a few examples (three per class, nine in total for our study) to the LLM as part of the prompt. This technique builds more contextual understanding before generating predictions. Tables 10 and 11 report the results obtained with few-shot prompting on the Gold and PRemo datasets, respectively, along with the relative change compared to the zero-shot baseline.

On the Gold dataset, the impact of few-shot prompting was comparatively limited, reflecting the already high baseline performance observed in the zero-shot setting. Macro-averaged F1-scores remained within a narrow range for most models, ranging between 65% and 85%, with improvements rarely exceeding 2–4 percentage points relative to zero-shot. This indicates that, in a controlled and less ambiguous dataset, providing additional in-context examples yields diminishing returns.

For the Positive class, performance was largely stable across models. Several models, including Gemma2 27B and Llama3.1 70B, achieved high F1-scores (85–86%) with only marginal changes compared to one-shot or zero-shot prompting. Precision and recall shifts were generally modest and often moved in opposite directions, suggesting that few-shot prompting mainly reshapes decision boundaries rather than introducing new discriminative capability in this setting. For the Negative class, few-shot prompting produced small and inconsistent effects. Some models, such as Gemma2 27B and GPT-4o, maintained high F1-scores close to 77–79%, but gains over zero-shot were marginal (3% for Gemma2 27B

Table 9 Performance with balanced one-shot prompting for the PRemo dataset (improvement vs baseline shown in parentheses)

Model	Class	Precision	Recall	F1-score
deepseek-r1-32b (n=1710)	Positive	74% (-12%)	75% (+9%)	74% (0%)
	Negative	75% (+4%)	40% (0%)	52% (+1%)
	Neutral	65% (+2%)	74% (-10%)	69% (-3%)
	Macro Avg.	71% (-2%)	63% (0%)	65% (-1%)
	Micro Avg.	69% (0%)	66% (-2%)	67% (-1%)
deepseek-r1-8b (n=1317)	Positive	80% (-4%)	40% (+6%)	54% (+5%)
	Negative	67% (+5%)	18% (-7%)	29% (-7%)
	Neutral	57% (+4%)	64% (-6%)	60% (0%)
	Macro Avg.	68% (+2%)	41% (-2%)	48% (-1%)
	Micro Avg.	63% (+4%)	46% (-3%)	53% (0%)
gemma2-27b (n=1790)	Positive	69% (+6%)	78% (-4%)	73% (+2%)
	Negative	70% (+3%)	59% (+8%)	64% (+6%)
	Neutral	71% (+2%)	71% (+6%)	71% (+4%)
	Macro Avg.	70% (+4%)	69% (+3%)	70% (+4%)
	Micro Avg.	70% (+4%)	70% (+4%)	70% (+4%)
gemma2-9b (n=1790)	Positive	62% (-14%)	83% (+15%)	71% (-1%)
	Negative	64% (-3%)	55% (+12%)	59% (+7%)
	Neutral	69% (+6%)	60% (-19%)	64% (-6%)
	Macro Avg.	65% (-4%)	66% (+3%)	65% (0%)
	Micro Avg.	66% (-2%)	65% (-2%)	66% (-2%)
gpt-4o (n=1791)	Positive	77% (-16%)	80% (+35%)	**79%** (+18%)
	Negative	68% (-9%)	**62%** (+22%)	65% (+13%)
	Neutral	**75%** (+16%)	77% (-16%)	**76%** (+4%)
	Macro Avg.	73% (-3%)	**73%** (+14%)	73% (+11%)
	Micro Avg.	74% (+8%)	**74%** (+8%)	**74%** (+8%)
gpt-4o-mini (n=1791)	Positive	68% (-13%)	82% (+17%)	75% (+2%)
	Negative	71% (-6%)	45% (+17%)	55% (+14%)
	Neutral	69% (+8%)	73% (-15%)	71% (-1%)
	Macro Avg.	69% (-4%)	67% (+6%)	67% (+5%)
	Micro Avg.	69% (+2%)	69% (+2%)	69% (+2%)
llama3.1-70b (n=1790)	Positive	67% (-6%)	84% (+7%)	75% (0%)
	Negative	**75%** (+6%)	52% (+12%)	61% (+11%)
	Neutral	72% (+6%)	72% (-5%)	72% (+1%)
	Macro Avg.	71% (+2%)	69% (+4%)	69% (+4%)
	Micro Avg.	71% (+2%)	71% (+2%)	71% (+2%)
llama3.1-8b (n=1791)	Positive	47% (-8%)	**92%** (+7%)	62% (-4%)
	Negative	63% (+3%)	52% (-4%)	57% (-1%)
	Neutral	72% (+2%)	36% (-13%)	48% (-10%)
	Macro Avg.	61% (-1%)	60% (-3%)	56% (-5%)
	Micro Avg.	56% (-5%)	56% (-5%)	56% (-5%)
mistral-nemo-12b (n=1786)	Positive	79% (+2%)	78% (+5%)	78% (+4%)
	Negative	71% (+5%)	61% (+11%)	**66%** (+9%)
	Neutral	73% (+7%)	78% (+2%)	75% (+5%)
	Macro Avg.	**74%** (+5%)	73% (+6%)	**73%** (+6%)
	Micro Avg.	**74%** (+5%)	74% (+5%)	74% (+5%)
mistral-small-22b (n=1790)	Positive	**85%** (-7%)	62% (+12%)	71% (+6%)
	Negative	73% (+2%)	52% (+11%)	61% (+9%)

Table 9 (continued)

Model	Class	Precision	Recall	F1-score
	Neutral	65% (+5%)	**85%** (-5%)	73% (+2%)
	Macro Avg.	74% (0%)	66% (+6%)	69% (+6%)
	Micro Avg.	70% (+4%)	70% (+4%)	70% (+4%)

and 0% for GPT-4o). In several cases, increased precision was accompanied by reduced recall, or vice versa, reinforcing that negative sentiment detection on the Gold dataset was already well captured by the baseline models. For the Neutral class, few-shot prompting occasionally led to increases in recall, particularly for models such as GPT-4o mini and Mistral Nemo, but these gains were often offset by drops in precision. As a result, F1-scores for neutral sentiment remained broadly stable, typically fluctuating within a 3% range relative to zero-shot.

Overall, the Gold dataset results suggest that few-shot prompting does not substantially improve performance beyond one-shot or zero-shot configurations when baseline accuracy is already high. The additional examples primarily redistribute errors across classes rather than delivering consistent gains, and in some cases introduce minor performance regressions.

On the PRemo dataset, for the Positive class, GPT-4o led with an F1-score of 80%, driven by a balanced precision (87%) and recall (74%). Mistral small achieved the highest precision overall (92%) but at the cost of recall (54%). Conversely, Llama3.1 8B demonstrated the highest recall (91%) but a low precision (50%), reflecting a tendency to over-label positives. For the Negative class, Gemma2 27B obtained the highest F1-score (62%), with recall (56%) surpassing that of larger models like GPT-4o (49%). This aligns with previous findings that negative sentiment detection benefits from explicit examples. Nevertheless, smaller models such as Deepseek-R1 8B performed poorly (F1-score of 22%), likely due to their limited memory or attention capacity, which can struggle to synthesize information from multiple examples effectively. For the Neutral class, we can see a shift, with GPT-4o reaching the top F1-score (78%) thanks to strong recall (89%), while Mistral small achieved the highest recall (91%) but with reduced precision (63%). This pattern indicates that few-shot prompting may help capture a broader range of neutral expressions, although precision trade-offs remain.

Overall, applying the few-shot technique generally improved model performance compared to zero-shot, particularly for challenging sentiment classes. The macro-averaged results show that GPT-4o achieved the highest F1-score (73%), followed closely by Mistral Nemo (71%) and Mistral Small (67%). Precision tended to be higher than recall for most models, suggesting that few-shot prompting helped models be more selective in their predictions but did not always boost recall equally across classes. Micro-averaged scores reveal a tighter clustering of results, with GPT-4o again leading (75%) and most other high-performing models achieving between 69% and 73%. Lower-capacity models, particularly Llama3.1 8B and Deepseek-R1 8B, struggled to leverage the few-shot context effectively, suggesting that model scale and architecture influence the benefits derived from prompt engineering. When comparing one-shot and few-shot results, a consistent trend emerges: *providing one example tends to yield broader and more stable improvements across sentiment classes*. While few-shot prompting offered gains over zero-shot, one-shot prompting generally produced more balanced trade-offs between precision and recall.

Table 10 Performance with few-shot prompting for the gold dataset (improvement vs baseline shown in parentheses)

Model	Class	Precision	Recall	F1-score
deepseek-r1-32b (n=7122)	Positive	92% (0%)	78% (-1%)	85% (0%)
	Negative	78% (-4%)	80% (+5%)	**79%** (+1%)
	Neutral	77% (+2%)	84% (-3%)	**81%** (0%)
	Macro Avg.	83% (-1%)	**81%** (0%)	**81%** (0%)
	Micro Avg.	**81%** (0%)	**81%** (0%)	**81%** (0%)
deepseek-r1-8b (n=7122)	Positive	86% (+10%)	80% (-5%)	83% (+2%)
	Negative	72% (-1%)	**80%** (+2%)	76% (0%)
	Neutral	78% (-2%)	75% (+6%)	77% (+3%)
	Macro Avg.	79% (+2%)	79% (+1%)	78% (+2%)
	Micro Avg.	78% (+2%)	78% (+2%)	78% (+2%)
gemma2-27b (n=7122)	Positive	88% (+7%)	83% (-8%)	85% (0%)
	Negative	77% (+11%)	77% (-8%)	77% (+3%)
	Neutral	**78%** (-8%)	82% (+19%)	80% (+8%)
	Macro Avg.	81% (+3%)	80% (+1%)	81% (+3%)
	Micro Avg.	81% (+4%)	81% (+4%)	81% (+4%)
gemma2-9b (n=7122)	Positive	83% (-5%)	**86%** (+6%)	85% (0%)
	Negative	73% (+6%)	74% (-4%)	74% (+2%)
	Neutral	77% (+3%)	75% (+4%)	76% (+3%)
	Macro Avg.	78% (+1%)	78% (+2%)	78% (+2%)
	Micro Avg.	78% (+2%)	78% (+2%)	78% (+2%)
gpt-4o (n=7122)	Positive	96% (0%)	64% (+2%)	77% (+1%)
	Negative	86% (+9%)	73% (-9%)	79% (0%)
	Neutral	70% (-2%)	**93%** (+8%)	80% (+2%)
	Macro Avg.	**84%** (+2%)	77% (0%)	78% (+1%)
	Micro Avg.	79% (+1%)	79% (+1%)	79% (+1%)
gpt-4o-mini (n=7122)	Positive	91% (-2%)	79% (+3%)	85% (+1%)
	Negative	**88%** (+8%)	65% (-9%)	75% (-2%)
	Neutral	72% (-1%)	91% (+6%)	80% (+1%)
	Macro Avg.	84% (+2%)	78% (0%)	80% (0%)
	Micro Avg.	80% (+1%)	80% (+1%)	80% (+1%)
llama3.1-70b (n=7122)	Positive	88% (+1%)	84% (-1%)	**86%** (0%)
	Negative	83% (+13%)	59% (-22%)	69% (-6%)
	Neutral	72% (-8%)	89% (+16%)	80% (+3%)
	Macro Avg.	81% (+2%)	77% (-2%)	78% (-1%)
	Micro Avg.	79% (0%)	79% (0%)	79% (0%)
llama3.1-8b (n=7121)	Positive	78% (+1%)	83% (-5%)	80% (-1%)
	Negative	74% (+14%)	58% (-26%)	65% (-4%)
	Neutral	69% (-12%)	76% (+27%)	73% (+11%)
	Macro Avg.	74% (+1%)	73% (-1%)	73% (+2%)
	Micro Avg.	73% (+2%)	73% (+2%)	73% (+2%)
mistral-nemo-12b (n=7110)	Positive	92% (0%)	73% (-2%)	81% (-1%)
	Negative	84% (+17%)	64% (-17%)	73% (-1%)
	Neutral	69% (-7%)	90% (+17%)	78% (+4%)
	Macro Avg.	82% (+4%)	76% (-1%)	77% (+1%)
	Micro Avg.	78% (+1%)	77% (+2%)	77% (+1%)
mistral-small-22b (n=7122)	Positive	**96%** (0%)	57% (-15%)	72% (-11%)
	Negative	82% (+7%)	65% (-16%)	72% (-6%)

Table 10 (continued)

Model	Class	Precision	Recall	F1-score
	Neutral	64% (-10%)	90% (+9%)	75% (-3%)
	Macro Avg.	81% (-1%)	71% (-8%)	73% (-6%)
	Micro Avg.	74% (-5%)	74% (-5%)	74% (-5%)

Finding 8: The few-shot approach compared to our baseline provided better performance for many models, especially in the harder, Negative class. However, the magnitude of improvement varied significantly across architectures. Larger models showed more effective results, while smaller models sometimes presented minimal or negative gains. Nevertheless, even for larger models, the performance of few-shot did not improve much when compared to one-shot, showing that there are diminishing returns to providing more examples.

5.3.3 Chain-of-thought (CoT) Technique

In this prompting technique, the model is guided to "think aloud" or generate intermediate reasoning steps before giving a final answer. This is possible by means of examples that provide this reasoning step (Diao et al. 2023). This can help with complex tasks by encouraging step-by-step analysis. We executed this technique with the same nine examples (with reasoning added) utilized in the few-shot technique. Tables 12 and 13 report the results obtained with one-shot prompting on the Gold and PRemo datasets, respectively, along with the relative change compared to the zero-shot baseline.

On the Gold dataset, Chain-of-Thought prompting led to relatively modest improvements, reflecting a ceiling effect similar to that observed with one-shot and few-shot techniques. Macro-averaged F1-scores increase modestly, typically between +1% and +6%, with most models remaining within the 73%-83% range. These gains are driven primarily by recall improvements, while precision frequently stagnates or decreases, indicating that CoT refines coverage rather than fundamentally improving class separability in a controlled setting.

GPT-4o benefits the most from CoT prompting. Its macro F1-score increases from 77% to 83% (+6%), with recall rising consistently across all classes, including a +12% increase for the Positive class and a +7% increase for the Neutral class. Importantly, these recall gains are achieved with only one mild case of precision degradation (-1% for the Positive class), making GPT-4o the strongest overall performer under CoT on this dataset. Other high-capacity models, such as Deepseek-R1 32B and Gemma2 27B, show smaller but consistent improvements. Deepseek-R1 32B achieves a macro F1-score of 83% (+1%), while Gemma2 27B reaches 81% (+3%), largely due to substantial recall gains for the Neutral class (+19%). However, these recall improvements are often offset by drops in precision for the same class, resulting in limited net gains.

Mid-sized models exhibit more mixed behaviour. GPT-4o mini improves marginally in macro F1-score (+1%), with increased recall for the Neutral class (+7%) but reduced recall for Positive and Negative sentiment. Llama3.1 70B maintains a stable macro F1-score (80%), but experiences strong class-level asymmetry, including a +17% recall increase for Neutral sentiment and a simultaneous -19% drop in recall for Negative sentiment. For

Table 11 Performance with few-shot prompting for the PRemo dataset (improvement vs baseline shown in parentheses)

Model	Class	Precision	Recall	F1-score
deepseek-r1-32b (n=1424)	Positive	72% (-13%)	57% (-9%)	64% (-11%)
	Negative	71% (0%)	34% (-5%)	47% (-4%)
	Neutral	62% (-1%)	59% (-24%)	61% (-11%)
	Macro Avg.	69% (-5%)	50% (-13%)	57% (-9%)
	Micro Avg.	66% (-3%)	53% (-15%)	59% (-10%)
deepseek-r1-8b (n=921)	Positive	80% (-4%)	28% (-6%)	42% (-7%)
	Negative	64% (+2%)	13% (-13%)	21% (-15%)
	Neutral	61% (+7%)	47% (-23%)	53% (-8%)
	Macro Avg.	68% (+2%)	29% (-14%)	39% (-10%)
	Micro Avg.	65% (+6%)	33% (-16%)	44% (-10%)
gemma2-27b (n=1790)	Positive	71% (+7%)	75% (-7%)	73% (+1%)
	Negative	70% (+2%)	**55%** (+4%)	**62%** (+3%)
	Neutral	69% (0%)	73% (+9%)	71% (+4%)
	Macro Avg.	70% (+3%)	68% (+2%)	68% (+3%)
	Micro Avg.	69% (+3%)	69% (+3%)	69% (+3%)
gemma2-9b (n=1790)	Positive	65% (-12%)	82% (+13%)	72% (0%)
	Negative	66% (-1%)	53% (+9%)	58% (+6%)
	Neutral	68% (+5%)	64% (-14%)	66% (-4%)
	Macro Avg.	66% (-3%)	66% (+3%)	66% (+1%)
	Micro Avg.	67% (-1%)	67% (-1%)	67% (-1%)
gpt-4o (n=1791)	Positive	87% (-6%)	73% (+28%)	**79%** (+19%)
	Negative	**77%** (0%)	48% (+9%)	60% (+7%)
	Neutral	69% (+10%)	89% (-4%)	**78%** (+5%)
	Macro Avg.	**78%** (+1%)	**70%** (+11%)	**72%** (+10%)
	Micro Avg.	**75%** (+8%)	**75%** (+8%)	**75%** (+8%)
gpt-4o-mini (n=1791)	Positive	72% (-10%)	79% (+13%)	75% (+2%)
	Negative	74% (-3%)	43% (+15%)	54% (+13%)
	Neutral	67% (+6%)	77% (-11%)	71% (-1%)
	Macro Avg.	71% (-2%)	66% (+6%)	67% (+5%)
	Micro Avg.	69% (+2%)	69% (+2%)	69% (+2%)
llama3.1-70b (n=1791)	Positive	74% (+1%)	80% (+2%)	77% (+2%)
	Negative	72% (+3%)	45% (+6%)	56% (+5%)
	Neutral	68% (+3%)	78% (+1%)	73% (+2%)
	Macro Avg.	72% (+2%)	68% (+3%)	69% (+3%)
	Micro Avg.	71% (+3%)	71% (+3%)	71% (+3%)
llama3.1-8b (n=1791)	Positive	49% (-6%)	**90%** (+5%)	64% (-3%)
	Negative	64% (+4%)	52% (-3%)	58% (0%)
	Neutral	**72%** (+2%)	41% (-8%)	52% (-5%)
	Macro Avg.	62% (0%)	61% (-2%)	58% (-3%)
	Micro Avg.	58% (-3%)	58% (-3%)	58% (-3%)
mistral-nemo-12b (n=1786)	Positive	79% (+3%)	76% (+3%)	78% (+3%)
	Negative	76% (+10%)	47% (-3%)	58% (+1%)
	Neutral	68% (+2%)	83% (+8%)	75% (+4%)
	Macro Avg.	75% (+5%)	69% (+3%)	70% (+3%)
	Micro Avg.	73% (+4%)	72% (+4%)	72% (+4%)
mistral-small-22b (n=1790)	Positive	**91%** (0%)	54% (+3%)	68% (+3%)
	Negative	75% (+3%)	46% (+5%)	57% (+5%)

Table 11 (continued)

Model	Class	Precision	Recall	F1-score
	Neutral	62% (+2%)	**90%** (+1%)	74% (+2%)
	Macro Avg.	76% (+2%)	63% (+3%)	66% (+3%)
	Micro Avg.	69% (+3%)	69% (+3%)	69% (+3%)

smaller models, CoT prompting introduces instability rather than consistent improvement. Llama3.1 8B shows recall gains for Positive (+1%) and Neutral (+14%) sentiment, but suffers recall losses for Negative (-13%), resulting in only a marginal macro F1 increase (+2%). Mistral Small is the most negatively affected by CoT, with its macro F1-score dropping from 80% to 74% (-6%), driven by substantial recall losses across Positive (-15%) and Negative (-17%) classes.

Overall, the Gold dataset results indicate that CoT prompting can improve recall and macro-level performance for high-capacity models, particularly GPT-4o, but offers diminishing returns once baseline performance is high. For smaller and some mid-sized models, the additional reasoning steps introduced by CoT often amplify class imbalance and can degrade overall performance.

Finding 9: CoT prompting yields modest and capacity-dependent gains on the Gold dataset, improving macro F1-scores by up to +6% primarily through recall increases for high-capacity models such as GPT-4o. However, these gains are constrained by ceiling effects, and for smaller or mid-sized models CoT often amplifies class imbalance or degrades performance.

On PRemo dataset, GPT-4o achieved the highest overall performance among all models, with a macro F1-score of 78% and balanced precision–recall trade-offs across all classes. It particularly excelled in the Negative class (F1-score of 71% and recall of 66%), where capturing subtle cues is typically more challenging. The most interesting example of this happened with the negative class, where most of the bigger models had a large improvement in metrics, especially in recall. In comparison with the zero-shot prompting, GPT-4o's recall in the Negative class improved 26%, allowing its F1-score to improve from 53% to 71%. Its smaller counterpart, GPT-4o mini, also maintained competitive macro-level performance (F1-score of 72%), suggesting that CoT reasoning steps may help even lighter versions of strong architectures retain contextual comprehension. Mid-sized models such as Mistral Small, Mistral Nemo, and Gemma2 27B showed moderate macro F1-scores (71–73%), with performance gains in specific classes.

On the other hand, certain models showed clear weaknesses when applying CoT. Deepseek-R1 8B performed poorly, with a dramatic drop in recall, particularly in the Positive (25%) and Negative (12%) classes, resulting in a low macro F1-score of 35%. It suggests that the additional reasoning overhead may have confused rather than clarified sentiment boundaries for some smaller architectures. Similarly, Llama3.1 8B suffered from extreme imbalance in class performance: while it achieved the highest recall in the Positive class (93%), its Neutral recall fell to 36%, resulting in low macro-level stability (F1-score of 57%). One possible reason for this is that, given the reduced memory footprint associated

Table 12 Performance with CoT prompting for the gold dataset (improvement vs baseline shown in parentheses)

Model	Class	Precision	Recall	F1-score
deepseek-r1-32b (n=7122)	Positive	93% (+1%)	79% (+1%)	86% (+1%)
	Negative	83% (+1%)	78% (+2%)	80% (+2%)
	Neutral	77% (+2%)	88% (+1%)	82% (+2%)
	Macro Avg.	84% (+1%)	82% (+1%)	83% (+1%)
	Micro Avg.	83% (+1%)	83% (+1%)	83% (+1%)
deepseek-r1-8b (n=7122)	Positive	88% (+11%)	82% (-3%)	85% (+4%)
	Negative	79% (+7%)	75% (-4%)	77% (+2%)
	Neutral	78% (-2%)	84% (+15%)	81% (+7%)
	Macro Avg.	82% (+5%)	80% (+3%)	81% (+4%)
	Micro Avg.	81% (+4%)	81% (+4%)	81% (+4%)
gemma2-27b (n=7122)	Positive	85% (+4%)	84% (-7%)	84% (-1%)
	Negative	78% (+12%)	76% (-9%)	77% (+3%)
	Neutral	**79%** (-7%)	82% (+19%)	80% (+8%)
	Macro Avg.	81% (+3%)	80% (+1%)	81% (+3%)
	Micro Avg.	81% (+4%)	81% (+4%)	81% (+4%)
gemma2-9b (n=7122)	Positive	85% (-3%)	83% (+2%)	84% (0%)
	Negative	72% (+4%)	74% (-3%)	73% (+1%)
	Neutral	75% (+1%)	74% (+4%)	75% (+2%)
	Macro Avg.	77% (+1%)	77% (+1%)	77% (+1%)
	Micro Avg.	77% (+1%)	77% (+1%)	77% (+1%)
gpt-4o (n=7122)	Positive	95% (-1%)	74% (+12%)	83% (+8%)
	Negative	87% (+10%)	**81%** (-1%)	**84%** (+5%)
	Neutral	77% (+5%)	92% (+7%)	**84%** (+6%)
	Macro Avg.	**86%** (+5%)	**82%** (+6%)	**83%** (+6%)
	Micro Avg.	**84%** (+6%)	**84%** (+6%)	**84%** (+6%)
gpt-4o-mini (n=7122)	Positive	94% (0%)	72% (-5%)	81% (-3%)
	Negative	**88%** (+9%)	74% (-1%)	80% (+3%)
	Neutral	73% (-1%)	**92%** (+7%)	81% (+2%)
	Macro Avg.	85% (+3%)	79% (0%)	81% (+1%)
	Micro Avg.	81% (+1%)	81% (+1%)	81% (+1%)
llama3.1-70b (n=7122)	Positive	90% (+2%)	84% (-2%)	**87%** (0%)
	Negative	84% (+14%)	62% (-19%)	72% (-4%)
	Neutral	73% (-7%)	90% (+17%)	81% (+4%)
	Macro Avg.	82% (+3%)	79% (-1%)	80% (0%)
	Micro Avg.	80% (+1%)	80% (+1%)	80% (+1%)
llama3.1-8b (n=7122)	Positive	73% (-4%)	**88%** (+1%)	80% (-2%)
	Negative	67% (+7%)	71% (-13%)	69% (-1%)
	Neutral	79% (-3%)	64% (+14%)	71% (+9%)
	Macro Avg.	73% (0%)	74% (+1%)	73% (+2%)
	Micro Avg.	73% (+2%)	73% (+2%)	73% (+2%)
mistral-nemo-12b (n=7115)	Positive	93% (+2%)	69% (-7%)	79% (-4%)
	Negative	80% (+13%)	66% (-15%)	72% (-1%)
	Neutral	68% (-7%)	88% (+16%)	77% (+3%)
	Macro Avg.	81% (+3%)	74% (-2%)	76% (0%)
	Micro Avg.	76% (0%)	76% (+1%)	76% (0%)
mistral-small-22b (n=7122)	Positive	**97%** (+1%)	58% (-15%)	73% (-10%)
	Negative	83% (+8%)	64% (-17%)	72% (-6%)

Table 12 (continued)

Model	Class	Precision	Recall	F1-score
	Neutral	64% (-9%)	92% (+10%)	76% (-2%)
	Macro Avg.	81% (0%)	71% (-7%)	74% (-6%)
	Micro Avg.	74% (-5%)	74% (-5%)	74% (-5%)

with those LLMs, they may struggle with maintaining coherence in multistep reasoning, resulting in no additional benefit from CoT techniques.

The application of the CoT prompting technique produced mixed, but generally positive, effects across models and sentiment classes. CoT appeared particularly beneficial for recall improvements, suggesting that explicit reasoning steps helped models capture a broader range of sentiment cues, even if it occasionally came at the cost of precision. For instance, GPT-4o demonstrated the most substantial macro-level gains, with recall rising by +17% and macro F1-score increasing by +16% compared to the zero-shot baseline. This was mirrored in class-level trends, such as a +35% recall boost for the Positive class and a +26% recall increase for the Negative class.

Finding 10: CoT can meaningfully improve performance in comparison to the zero-shot prompting, particularly for high-capacity and mid-capacity models, by enhancing recall and better capturing subtle polarity cues. However, for Deepseek-R1 8B, the step-by-step reasoning can introduce over-interpretation errors.

In summary, while prompt engineering improves zero-shot LLM performance in some settings, it is important to note that none of the prompting techniques consistently close the gap with the fine-tuned transformer baselines reported in Section 5.1, particularly on the Gold dataset.

Nevertheless, when comparing the four prompting techniques — zero-shot, one-shot, few-shot, and CoT — clear patterns emerge regarding their relative effectiveness across models and sentiment classes. For zero-shot, while some high-capacity models (e.g., GPT-4o, Gemma2 27B) achieved reasonable performance, they generally struggled in the Negative and Neutral classes, a finding consistent with prior research (Lin et al. 2018; Islam and Zibran 2017) that highlights the difficulty of detecting subtle or implicitly expressed sentiment. One-shot prompting leads to notable relative improvements, especially in the Negative class for models such as Mistral Small and Gemma2 27B. Few-shot prompting (with multiple examples) generally produced more stable and balanced performance across classes. Models like GPT-4o and Mistral Nemo benefited from the richer contextual grounding, achieving consistent precision-recall trade-offs and reducing extreme variability between classes.

Nonetheless, for certain mid-sized models, the improvement over one-shot was marginal, suggesting diminishing returns. The CoT technique yielded the highest overall performance for some large models, particularly boosting recall in harder-to-detect classes like Neutral and Negative. However, for some models, such as Deepseek-R1 8B, CoT showed substantial precision or recall drops, likely due to the reasoning steps amplifying irrelevant details or creating noise that obscured sentiment cues. These findings suggest that prompt selection should be tailored to the model's scale and the complexity of the sentiment detection task.

Table 13 Performance with CoT prompting for the PRemo dataset (improvement vs baseline shown in parentheses)

Model	Class	Precision	Recall	F1-score
deepseek-r1-32b (n=1688)	Positive	74% (-12%)	80% (+15%)	77% (+3%)
	Negative	71% (0%)	59% (+20%)	65% (+14%)
	Neutral	72% (+8%)	65% (-19%)	68% (-4%)
	Macro Avg.	72% (-1%)	68% (+5%)	70% (+4%)
	Micro Avg.	72% (+3%)	68% (0%)	70% (+2%)
deepseek-r1-8b (n=720)	Positive	82% (-2%)	25% (-9%)	38% (-11%)
	Negative	68% (+5%)	11% (-15%)	19% (-17%)
	Neutral	65% (+11%)	37% (-33%)	47% (-13%)
	Macro Avg.	72% (+5%)	24% (-19%)	35% (-14%)
	Micro Avg.	69% (+10%)	27% (-22%)	39% (-14%)
gemma2-27b (n=1790)	Positive	73% (+10%)	78% (-4%)	75% (+4%)
	Negative	70% (+2%)	64% (+13%)	67% (+9%)
	Neutral	73% (+4%)	73% (+8%)	73% (+6%)
	Macro Avg.	72% (+5%)	72% (+6%)	72% (+6%)
	Micro Avg.	72% (+6%)	72% (+6%)	72% (+6%)
gemma2-9b (n=1790)	Positive	62% (-14%)	85% (+17%)	72% (0%)
	Negative	66% (-1%)	51% (+7%)	57% (+5%)
	Neutral	69% (+6%)	61% (-17%)	65% (-5%)
	Macro Avg.	66% (-3%)	66% (+2%)	65% (0%)
	Micro Avg.	66% (-1%)	66% (-1%)	66% (-1%)
gpt-4o (n=1791)	Positive	86% (-7%)	80% (+35%)	**83%** (+22%)
	Negative	77% (0%)	**66%** (+26%)	**71%** (+19%)
	Neutral	75% (+16%)	84% (-9%)	**79%** (+7%)
	Macro Avg.	**79%** (+3%)	**76%** (+17%)	**78%** (+16%)
	Micro Avg.	**78%** (+12%)	**78%** (+12%)	**78%** (+12%)
gpt-4o-mini (n=1791)	Positive	75% (-6%)	83% (+18%)	79% (+7%)
	Negative	**79%** (+2%)	48% (+20%)	59% (+19%)
	Neutral	70% (+9%)	80% (-9%)	75% (+2%)
	Macro Avg.	75% (+2%)	70% (+10%)	71% (+9%)
	Micro Avg.	73% (+6%)	73% (+6%)	73% (+6%)
llama3.1-70b (n=1791)	Positive	68% (-5%)	86% (+9%)	76% (+1%)
	Negative	70% (+1%)	52% (+12%)	60% (+9%)
	Neutral	72% (+7%)	70% (-7%)	71% (0%)
	Macro Avg.	70% (+1%)	69% (+5%)	69% (+4%)
	Micro Avg.	70% (+2%)	70% (+2%)	70% (+2%)
llama3.1-8b (n=1791)	Positive	47% (-7%)	**93%** (+8%)	63% (-4%)
	Negative	62% (+2%)	56% (0%)	58% (+1%)
	Neutral	**79%** (+8%)	35% (-13%)	49% (-9%)
	Macro Avg.	62% (+1%)	61% (-2%)	57% (-4%)
	Micro Avg.	57% (-4%)	57% (-4%)	57% (-4%)
mistral-nemo-12b (n=1785)	Positive	76% (-1%)	79% (+6%)	77% (+3%)
	Negative	68% (+1%)	**66%** (+16%)	67% (+10%)
	Neutral	74% (+8%)	72% (-3%)	73% (+3%)
	Macro Avg.	72% (+3%)	72% (+6%)	72% (+5%)
	Micro Avg.	73% (+4%)	73% (+4%)	73% (+4%)
mistral-small-22b (n=1790)	Positive	**90%** (-1%)	58% (+8%)	71% (+6%)
	Negative	75% (+4%)	57% (+16%)	65% (+13%)

Table 13 (continued)

Model	Class	Precision	Recall	F1-score
	Neutral	65% (+6%)	**88%** (-2%)	75% (+3%)
	Macro Avg.	77% (+3%)	68% (+7%)	70% (+7%)
	Micro Avg.	72% (+5%)	72% (+5%)	72% (+5%)

Finding 11: The results indicate that prompting techniques interact strongly with model capacity. Zero-shot is marginally the faster than other techniques (due to having less input tokens to process) but less reliable for nuanced classes. One-shot often does not bring meaningful gains. Few-shot offers more stable and balanced performance. CoT can push large models to peak results but may harm smaller models. Therefore, as more complex prompt engineering techniques are employed, the boost to bigger models becomes larger. Conversely, fewer benefits could be observed for smaller models, and, in some cases, they became worse than zero-shot.

5.4 Qualitative Analysis

To deepen our understanding of why our best-performing model (GPT-4o) misclassified certain messages, we conducted a qualitative analysis (see Step 5 in Section 4.2) through an open coding process focused on highlighting characteristics of messages that may have affected the model's performance. The codes and their descriptions can be seen in Table 14. To assess inter-rater reliability, we computed Cohen's Kappa between the two primary coders prior to adjudication. Agreement was substantial (Cohen's $\kappa \approx 0.79$), indicating strong consistency beyond chance. Disagreements were subsequently resolved through adjudication by a third researcher, which was required in 24 cases, corresponding to 21.05% of the analyzed sample. A per-code breakdown of inter-rater agreement, including Cohen's Kappa values and code frequencies, is reported in the supplementary material.

In total, 277 codes were assigned across the sample of 114 messages, since some categories were present in multiple messages. *Technical Terms* appeared in more than half of the sample (60), suggesting that highly domain-specific vocabulary poses a persistent challenge for LLMs in sentiment interpretation - likely due to overlapping technical and emotional semantics. *Sentiment-Charged Keywords* was found in 41 instances, highlighting how terms such as "fail", "error", or "bug" may carry emotionally suggestive connotations while conveying different meanings based on context. *Sentiment Cue at Start* was found in 27 instances. It suggests positional bias in the model's attention, where initial emotive cues may disproportionately influence sentiment attribution, potentially overshadowing more neutral or corrective content later in the message. The *Long Messages* code was found in 21 instances. It shows possible ambiguity, as long messages can include multiple sentiment indicators within a single message. Some of these indications could also be peripheral and may not reflect the core sentiment of the message, thus misleading the model.

These characteristics are also consistent with the sharp recall degradation observed for transformer-based baselines on the PRemo dataset, suggesting that both fine-tuned and prompt-based approaches struggle with similar sources of ambiguity.

Table 14 Codes assigned during our qualitative analysis, along with their descriptions and number of occurrences

Code	Definition	Instances
Technical Terms	Technical terms may cause sentiment misinterpretation if context is unclear.	60
Sentiment-Charged Keywords	Strongly connotative words (e.g., "error", "amazing") can skew model sentiment classification, overshadowing overall context.	41
Sentiment Cue at Start	Initial sentiment cues can bias model classification by overshadowing full context.	27
Long message	Long messages can confuse AI with excessive information (e.g., code snippets) that humans easily ignore, leading to misinterpretation.	21
Error Reporting	Error descriptions may be seen as negative by the model, regardless of neutral/constructive tone.	20
Suggestion or Instruction	Action-directing phrases (e.g., "please do this") have tone-dependent sentiment, ranging from neutral/positive to negative if critical/assertive.	20
Links	Links may add noise, skewing sentiment analysis.	14
Expression of Uncertainty	Hesitant tone in phrases of doubt (e.g., "it seems") may result in model classifying sentiment as neutral or slightly negative.	13
Use of Emoji	Emojis add emotional/contextual nuances (e.g., humor, emphasis) that models may misinterpret or overlook, causing inaccurate sentiment classification.	11
Code Snippets in Messages	Code snippets/references can shift model focus to technical content, reducing sentiment sensitivity.	10
Use of Slang	Slang/informal phrases (e.g., "LGTM", "IMO") can confuse the model's interpretation of tone and context.	9
Message-in-Message Structure	Embedded structures (e.g., quotes, headers) can confuse the model with irrelevant context, causing sentiment misclassification.	7
Ambiguous Tone	Informal phrases may obscure tone, complicating model's sentiment analysis.	5
Lack of Context Awareness	Lack of context in isolated sentence analysis hinders full understanding of message intent.	5
Correction as a Question	Model may overlook corrective intent in question-phrased corrections.	4
Use of Sarcasm	Difficulty detecting sarcasm (e.g., "Great, another bug") leads models to misinterpret sentiment, often as positive or neutral.	4
Quoted Words	Quotation marks can add emphasis or imply nuances (sarcasm, uncertainty) that may lead to model misinterpreting sentiment/tone.	2
Use of Exclamation Marks	Exclamation marks amplify emotional tone (excitement, urgency, frustration), potentially causing model sentiment misclassification due to perceived intensity.	2
Ambiguity in Word Meaning	Words with multiple meanings (e.g., "run," "debug") can confuse the model without context, affecting sentiment classification accuracy.	1
Use of Caps Lock	Capitalization (e.g., "ERROR") can imply strong emotions, potentially causing models to overestimate sentiment intensity.	1

This coding process reveals that sentiment misclassifications are not isolated to sentiment polarity confusion but often stem from the model's difficulty in resolving semantic nuance, contextual framing, and rhetorical function. Together, these findings underscore the limitations of LLMs when applied to domain-specific and pragmatically complex communication. Addressing these challenges may require targeted strategies such as enhanced domain adaptation (e.g., fine-tuning), or hybrid approaches that combine LLMs with rule-based post-processing to filter out spurious sentiment triggers.

The complete spreadsheet with the sample messages, their associated codes, and explanations for each code is available in our replication package (Coutinho et al. 2025a).

Finding 12: Misclassifications in PR discussions predominantly arise from the presence of Technical Terms (60), Sentiment-Charged Keywords (41), Sentiment Cue at Start (27), and Long Messages (21), rather than from arbitrary polarity confusion.

6 Implications

This study systematically evaluates the efficacy of ten LLMs for sentiment analysis of GitHub PR discussions, examining their performance across multiple evaluation metrics. We also investigate how three distinct prompt engineering techniques influence model accuracy and reliability. The findings reveal significant performance variations between models and demonstrate how strategic prompt design can substantially improve sentiment classification results. These insights provide actionable guidance for developers integrating sentiment analysis into code review workflows, tool builders designing automated moderation systems, and researchers advancing natural language processing applications in software development contexts.

Implications for GitHub Users and Practitioners For GitHub users and organizations interested in applying sentiment analysis to pull request discussions, our results suggest that tool selection should be guided by the intended use case rather than by overall accuracy alone. While LLMs consistently outperform traditional sentiment analysis tools, the absolute performance remains moderate, indicating that no current approach provides fully reliable sentiment classification in this setting. Consequently, sentiment analysis tools should be used as decision-support mechanisms rather than as definitive or fully automated solutions. For example, organizations aiming to monitor community health or identify potentially problematic interactions may benefit from LLM-based approaches as a first-pass filter, while still relying on human oversight for interpretation and intervention.

The qualitative findings further contextualize these limitations by revealing that many misclassifications stem from recurring linguistic and pragmatic features of PR discussions, rather than from random model errors. The prevalence of *Technical Terms*, *Sentiment-Charged Keywords*, and *Long Messages* helps explain why sentiment analysis tools—despite strong overall language understanding—struggle to reliably detect polarity in real-world developer communication. For practitioners, this implies that lower recall or precision values observed in quantitative evaluations are often tied to systematic properties of the data, such as indirect criticism or multi-purpose technical statements, rather than deficiencies in a specific model. As a result, performance metrics should be interpreted as indicators of expected uncertainty rather than as absolute measures of correctness.

LLMs can Outperform Traditional Tools Our evaluation demonstrates that LLMs consistently outperform traditional and domain-specific sentiment analysis tools commonly used in software engineering contexts (see Section 5.2). This performance advantage may be a result of LLMs' ability to interpret the contextual and technical nuances inherent in software development discussions. These findings establish LLMs as a strong choice for polarity-based senti-

ment analysis in collaborative development platforms. However, traditional tools may still be viable in scenarios where computational resources, privacy constraints, or deployment simplicity are primary concerns. Thus, the choice between SentiStrength-like tools and LLM-based approaches involves a trade-off between performance gains and operational cost.

The qualitative analysis provides further insight into why LLMs outperform traditional tools, particularly for negative sentiment. Traditional lexicon-based or shallow learning approaches are especially sensitive to *Sentiment-Charged Keywords* (e.g., "error", "fail", "broken"), which frequently appear in neutral or constructive technical contexts. In contrast, LLMs are better equipped to partially disambiguate these terms by considering surrounding context, rhetorical intent, and discourse structure. Nevertheless, our qualitative results show that even LLMs remain vulnerable when technical terminology overlaps strongly with affective language, explaining why negative sentiment recall remains a persistent challenge across all evaluated approaches.

It is important to note, however, that this advantage does not extend uniformly across all modern sentiment analysis approaches. In particular, fine-tuned transformer-based models achieved substantially higher performance on the Gold dataset, reaching excellent macro F1-scores (91–92%), and in some cases exhibited more reliable recall for negative sentiment than zero-shot or prompt-based LLMs. This indicates that, when sufficient labeled data is available and domain characteristics are well controlled, supervised transformer-based methods can still outperform LLMs. The observed superiority of LLMs over traditional tools therefore reflects a comparison with non–fine-tuned, lexicon-based, or shallow learning approaches, rather than with fully supervised state-of-the-art baselines.

No Model is Universally Better While GPT-4o achieved the strongest overall performance, no single model excelled across all evaluation dimensions. Each model exhibited distinct strengths for different sentiment classes (positive, negative, neutral), suggesting the importance of aligning model selection with task-specific requirements and operational constraints—whether prioritizing toxicity detection, sentiment-based task assignment, or other specialized needs (see Section 5.2).

In particular, use cases such as toxicity monitoring, moderation support, or identification of problematic interactions require careful distinction between sentiment and toxicity. Prior research has shown that negative sentiment does not necessarily indicate abusive or toxic behaviour, especially in software engineering discussions where criticism and disagreement may be expressed constructively (Sayago-Heredia et al. 2022). While sentiment analysis and toxicity detection are therefore related but fundamentally distinct tasks, sentiment signals may nevertheless serve as an auxiliary or first-level indicator to highlight interactions that warrant further scrutiny. Under this exploratory perspective, models with stronger performance on negative sentiment may be useful for surfacing potentially sensitive discussions.

The selection process must also weigh privacy and scalability considerations. Open source models delivered competitive performance while offering superior cost efficiency and deployment flexibility. Their capacity for local deployment addresses critical data privacy requirements for organizations handling sensitive information. Conversely, proprietary models like GPT-4o provide streamlined cloud-based deployment with minimal hardware

requirements, making them ideal for scenarios where data privacy concerns are mitigated through preprocessing steps like anonymization. Organizations should evaluate whether cloud-hosted open-source models might optimize both performance and resource utilization for their specific contexts. Regardless of deployment strategy, practitioners should remain aware that sentiment-based signals are best used in conjunction with additional contextual or behavioral indicators, particularly for moderation-related applications (Sayago-Heredia et al. 2022; Brassard-Gourdeau and Khoury 2018, 2019).

Importantly, the qualitative results suggest that these class-level trade-offs are not merely statistical artifacts but reflect deeper modeling challenges. For instance, the frequent occurrence of *Sentiment Cue at Start* indicates that models may overweight early lexical signals, leading to premature polarity assignments that are not revised when later content reframes the message. This behavior aligns with the observed tendency, across both LLMs and transformer-based baselines, to over-predict neutral sentiment or misclassify negative sentiment as neutral on the PRemo dataset. Consequently, selecting a model with higher recall for negative sentiment may reduce missed cases, but it does not eliminate the underlying ambiguity inherent in PR discourse.

Subjectivity, Dataset Characteristics, and Performance Ceilings The relatively weak performance observed across both LLM-based and traditional tools must be interpreted in light of the inherent subjectivity of sentiment annotation and the structural characteristics of the datasets. While the Gold dataset yields excellent performance for fine-tuned transformer models (macro F1-scores of 91–92%), this should be viewed as an upper-bound scenario: as the data is more abundant (more messages) and shorter on average than the messages in PRemo.

The qualitative analysis helps explain this performance gap. Codes such as *Technical Terms* and *Long Messages* are far more prevalent in PR discussions and directly contribute to the recall degradation observed for both transformer-based models and LLMs on PRemo. These findings indicate that the drop in quantitative performance is not primarily due to overfitting or model weakness, but rather to an increase in semantic ambiguity and rhetorical complexity. As a result, the strong results on the Gold dataset should be interpreted as demonstrating what is achievable under more controlled conditions, whereas performance on PRemo more accurately reflects the practical limits of sentiment analysis in software engineering contexts, especially in more chaotic scenarios where curation is not possible.

Prompt-Engineering Techniques are more Effective on Larger Models Prompt engineering techniques, particularly Chain-of-Thought (CoT) reasoning, substantially enhanced performance metrics for larger models like GPT-4o. These techniques enhance contextual reasoning and comprehension capabilities, but smaller models showed minimal improvement, suggesting that the complexity of prompts should align with the capacity of the models. This insight is particularly valuable for practitioners seeking to optimize model efficiency and cost, as we have observed, as in the case with GPT-4o and GPT-4o-mini, that differences in results may not be as significant as the difference in cost between models.

Taken together, the qualitative and quantitative findings suggest that many observed gains from prompt engineering, particularly improvements in recall for negative sentiment, arise from helping models better navigate these sources of ambiguity rather than from improv-

ing raw sentiment discrimination. Prompt examples and reasoning steps appear to provide additional anchoring signals that partially counteract misleading lexical cues or diffuse sentiment spread across long messages. However, as shown by the transformer-based baselines on the Gold dataset, where there are potentially less sentiment cues, sophisticated prompting becomes unnecessary, reinforcing that prompt engineering primarily compensates for data complexity rather than replacing domain adaptation.

7 Threats to Validity

We discuss threats to the validity of the study following the taxonomy proposed by Wohlin et al. (2012).

7.1 Construct Validity

Model Coverage and Prompting Technique Limitations Our study examined only a subset of available LLMs, which may not represent the full spectrum of model capabilities. Other LLMs or different configurations of the selected models could yield superior or inferior performance compared to our findings. Additionally, newer or more advanced models may have been released since the completion of this study, potentially affecting the generalizability of our results. We evaluated a limited set of prompting techniques, which may not capture the optimal performance potential of each model. Different prompt engineering approaches or more sophisticated prompting strategies could lead to improved results beyond what we observed. For example, we did not assess inference-time ensembling techniques such as self-consistency (i.e., multiple reasoning samples with majority voting).

7.2 Internal Validity

Dataset Labeling Bias The data utilized in this work was manually labeled by humans. The steps taken to create the dataset might introduce bias to this work. To mitigate this, we selected a dataset that has taken several measures to reduce bias, including expert consensus on labels, literature-based labeling frameworks, and other bias-reduction techniques. However, inherent subjectivity in human annotation remains a limitation.

Example Selection Bias The selection of classification examples and their corresponding rationales for few-shot and chain-of-thought prompting may introduce researcher bias. This is a significant threat because these prompting techniques are highly sensitive to the provided examples. Poorly chosen, or unrepresentative examples could inadvertently steer the model toward a flawed understanding of the task, leading to skewed or inaccurate classifications. To mitigate this, we addressed this concern by having two authors collaboratively select and review all examples and rationales through a paired validation process. While this approach helps ensure the examples are balanced and clear, reducing the impact of any single researcher's potential biases, representativeness still remains an issue (albeit a smaller one) due to the limited number of examples.

Model Quantization Effects Due to hardware constraints, for the models that were locally executed via Ollama, we executed quantized versions of the models. While quantization reduces computational requirements, it may impact model quality. We employed Ollama's default Q4_0 quantization, which balances performance and scalability and represents typical user configurations. Nevertheless, results may differ when using full versions of the models. Additionally, all experiments were conducted with `temperature = 0` and a single prompt instantiation per configuration. Although this improves reproducibility, results may differ in less deterministic contexts. In such cases, additional techniques which were not used in our experimental setup, such as majority voting using multiple prompts, models or multiple executions of the same model could be used.

7.3 Conclusion Validity

Statistical Analysis Limitation Our analysis focused on comparative performance trends based on precision, recall and F1-score metrics. We did not conduct formal statistical significance testing (e.g., bootstrap confidence intervals, McNemar or Bowker tests) to assess whether observed differences between models or prompting techniques are statistically significant or to quantify their effect sizes. As a result, conclusions regarding relative performance, particularly when differences between methods are numerically small, should be interpreted as indicative rather than definitive. Future work should incorporate statistical testing and uncertainty estimation methods to assess the robustness and practical relevance of performance differences, particularly when models exhibit similar aggregated scores.

7.4 External Validity

Limited Dataset Scope Our evaluation relies on two datasets derived from GitHub PR discussions, the Gold dataset and the PRemo dataset, encompassing various software projects in total. While the use of two independently curated datasets enables cross-dataset comparison and mitigates some dataset-specific biases, the scope of the evaluation remains limited to PR discussions on a single platform. As a result, their generalizability of our findings to other SE communication contexts (e.g., issues, chats, code reviews outside GitHub), or to non-software engineering domains is constrained.

Context Specificity The results are specific to sentiment analysis in software development communications and may not extend to sentiment analysis tasks in other domains or applications. The unique characteristics of developer discourse and technical communication may not be representative of broader challenges in sentiment analysis.

8 Conclusion and Future Work

This study evaluates the effectiveness of current LLMs for sentiment analysis in GitHub PR discussions, contributing to the research on how modern AI approaches can enhance software development workflows. Through systematic experimentation with multiple LLMs and prompt engineering techniques, we demonstrate substantial performance gains over traditional sentiment analysis tools commonly deployed in software engineering contexts.

These advancements demonstrate the suitability of LLMs for tasks that require interpreting complex contextual and technical nuances present in software development discussions. We found that no single LLM universally outperforms others across all metrics or task requirements. Instead, each model exhibits distinct strengths across different sentiment classes (positive, negative, neutral), emphasizing the need for developers to select models based on specific task needs, such as toxicity detection or the prioritization of sentiment-driven assignments, while also considering resource constraints.

This paper contributes to the state-of-the-art by providing actionable implications for developers, tool builders, and researchers regarding the selection and deployment of LLMs for sentiment analysis. We also discuss the trade-offs between the proprietary and open-source models tested. Furthermore, our study demonstrates that prompt engineering techniques, particularly the Chain-of-Thought (CoT) approach, lead to performance improvements in sentiment analysis tasks, especially in larger models. Conversely, smaller models showed limited benefits from increased prompt complexity, suggesting that prompt design should be aligned with the model's capacity. These findings serve as a valuable reference for building scalable and task-specific sentiment analysis tools.

Beyond aggregate performance metrics, our qualitative analysis provides critical insight into why strong quantitative results do not always translate into reliable real-world behavior. While large models and advanced prompting techniques (e.g., CoT) can improve recall and macro-level performance, as we have seen particularly on the PRemo dataset, misclassifications persist due to linguistic and pragmatic factors that are not fully captured by standard evaluation metrics. Specifically, errors are frequently driven by the presence of technical terminology, sentiment-charged keywords with domain-specific meanings (e.g., "fail", "error"), early positional sentiment cues, and long messages containing mixed or shifting sentiment. These findings help explain the systematic recall degradation for negative sentiment observed quantitatively: many messages labeled as neutral or factual still express implicit criticism that models struggle to disambiguate. Thus, the qualitative results contextualize the observed precision/recall trade-offs, showing that performance limitations stem less from random noise and more from recurring challenging discourse patterns in PR discussions. This contrast highlights an important limitation of relying solely on aggregate metrics and underscores the need for domain-aware modeling strategies that explicitly address these sources of ambiguity.

An important direction for future work is the fine-tuning and domain adaptation of open-source LLMs for sentiment analysis in software engineering contexts. While this study focused on out-of-the-box model performance to reflect realistic deployment scenarios, our findings can directly inform fine-tuning efforts. For instance, the observed class-level weaknesses - particularly for negative sentiment and recall on the PRemo dataset - provide clear targets for supervised or instruction-based fine-tuning.

Another important direction for future work is the evaluation of inference-time ensembling strategies, such as self-consistency through majority voting over multiple reasoning paths. While this study demonstrates that CoT prompting yields meaningful gains under single-pass, `temperature=0` inference, additional improvements may be attainable by aggregating multiple, varying, CoT prompts.

Future studies could leverage curated datasets such as the Gold dataset for controlled fine-tuning, followed by evaluation on more ambiguous, real-world datasets like PRemo to assess generalization and robustness. Parameter-efficient fine-tuning techniques (e.g., LoRA

or adapter-based methods) represent a promising avenue to improve performance while preserving the deployment advantages of open-source models. Additionally, fine-tuning strategies could be combined with prompt engineering, enabling systematic analysis of how training-time adaptation and inference-time prompting interact. Such investigations would further clarify the performance ceiling observed in this work and help determine to what extent it can be overcome through model adaptation.

An important direction for future work is the systematic evaluation of robustness in LLM-based sentiment analysis. Future studies should assess model sensitivity to realistic perturbations commonly found in PR discussions, such as typos, informal language, code snippets, distractor text, and paraphrasing. Controlled robustness analyses would complement the results presented in this work and help determine how stable different prompting strategies and model architectures are under noisy real-world conditions.

Another relevant direction for future work concerns context-aware sentiment analysis in PR discussions. Although this study adopts a message-level formulation for consistency with prior work, future studies could explicitly investigate the impact of conversational context by incorporating one or more preceding messages from the same thread into the prompt or training setup. Ablation experiments comparing message-only versus context-aware configurations would help clarify how much contextual information contributes to performance gains, particularly for ambiguous or mixed-sentiment messages. Such analyses could be naturally combined with fine-tuning strategies to assess whether contextual modeling yields consistent improvements across datasets.

Author Contributions Daniel Coutinho: Conceptualization, Study Design, Data Collection, Data Analysis, Manuscript Writing, Review. Breno Braga: Data Collection, Data Analysis, Manuscript Writing. Theo Canuto: Data Collection, Data Analysis, Manuscript Writing. Juliana Alves Pereira: Study Design, Manuscript Writing, Critical Review. Wesley K.G. Assunção: Study Design, Critical Review. Igor Steinmacher: Study Design, Critical Review. Marco Gerosa: Study Design, Critical Review. Alessandro Garcia: Study Design, Critical Review.

Funding The Article Processing Charge (APC) for the publication of this research was funded by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) (ROR identifier: 00x0ma614). This research was partially funded by the Brazilian funding agencies CAPES (88881.879016/2023-01), CAPES/Procad (175956), CAPES/PROEX, CNPq (434969/2018-4, 312149/2016-6, 141180/2021-8, GD), FAPESP (2023/00811-0), FAPERJ (200773/2019, 010002285/2019), NSF (2303042, 2247929, 2303612), and the Binational Cooperation Program CAPES/COFECUB (Ma1036/24). We also acknowledge the Brazilian company Stonehttps://www.stone.com.br/ for their support.

Data Availability Statement All available data are currently available at Coutinho et al. (2025a).

Declarations

Ethical Approval This item is not applicable.

Informed Consent This item is not applicable.

Conflict of Interest The authors declare that they have no conflict of interest.

Clinical Trial Number This item is not applicable.

References

Ain QT, Ali M, Riaz A, Noureen A, Kamran M, Hayat B, Rehman A (2017) Sentiment analysis using deep learning techniques: a review. Intern J Adv Comput Sci Appl 8(6) https://doi.org/10.14569/IJACSA.2017.080657

Barbosa C, Uchôa A, Coutinho D, Assunçao WK, Oliveira A, Garcia A, Fonseca B, Rabelo M, Coelho JE, Carvalho E et al (2023) Beyond the code: Investigating the effects of pull request conversations on design decay. In: 2023 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM), IEEE, pp 1–12 https://doi.org/10.1109/ESEM56168.2023.10304805

Brassard-Gourdeau É, Khoury R (2018) Impact of sentiment detection to recognize toxic and subversive online comments. arXiv preprint arXiv:1812.01704

Brassard-Gourdeau E, Khoury R (2019) Subversive toxicity detection using sentiment information. In: Proceedings of the third workshop on abusive language online, pp 1–10 https://doi.org/10.18653/v1/W19-3501

Brown T, Mann B, Ryder N, Subbiah M, Kaplan JD, Dhariwal P, Neelakantan A, Shyam P, Sastry G, Askell A et al (2020) Language models are few-shot learners. Adv Neural Inf Process Syst 33:1877–1901

Calefato F, Lanubile F, Novielli N (2017) Emotxt: A toolkit for emotion recognition from text. In: Proceedings of the seventh international conference on affective computing and intelligent interaction, IEEE, pp 79–85 https://doi.org/10.1109/ACIIW.2017.8272591

Calefato F, Lanubile F, Maiorano F, Novielli N (2018) Sentiment polarity detection for software development. In: Proceedings of the 40th international conference on software engineering, IEEE, pp 128-128 https://doi.org/10.1145/3180155.3182519

Coutinho D, Cito L, Lima MV, Arantes B, Pereira JA, Arriel J, Godinho J, Martins V, Libório P, Leite L, Garcia A, Assunção WKG, Steinmacher I, Baffa A, Fonseca B (2024) "looks good to me ;-)": Assessing sentiment analysis tools for pull request discussions. In: Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering (EASE 2024), ACM, Salerno, Italy, p 11. https://doi.org/10.1145/3661167.3661189

Coutinho D, Braga B, Canuto T, Pereira JA, Assunção WKG, Steinmacher I, Gerosa M, Garcia A (2025a) Replication package. https://github.com/opus-research/LLM4SA-replication

Coutinho D, Pereira JA, Neves B, Correia JLM, da Silva CBV, Assunção WKG, Steinmacher I, Gerosa MA, Baffa A, Garcia A (2025b) PRemo: A Dataset of Emotions Found on Pull Request Discussions. In: Proceedings of the 39th Brazilian Symposium on Software Engineering (SBES), SBC, Brazil, to appear https://doi.org/10.5753/sbes.2025.9936

Dang NC, Moreno-García MN, De la Prieta F (2020) Sentiment analysis based on deep learning: A comparative study. Electronics 9(3):483 https://doi.org/10.3390/electronics9030483

Devlin J, Chang MW, Lee K, Toutanova K (2018) Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805

Diao S, Wang P, Lin Y, Pan R, Liu X, Zhang T (2023) Active prompting with chain-of-thought for large language models. arXiv preprint arXiv:2302.12246

El Mezouar M, Zhang F, Zou Y (2019) An empirical study on the teams structures in social coding using github projects. Empir Softw Eng 24(6):3790–3823 https://doi.org/10.1007/s10664-019-09700-1

Graziotin D, Wang X, Abrahamsson P (2014) Happy software developers solve problems better: psychological measurements in empirical software engineering. PeerJ 2:e289 https://doi.org/10.7717/peerj.289

Graziotin D, Wang X, Abrahamsson P (2015) How do you feel, developer? an explanatory theory of the impact of affects on programming performance. PeerJ Comput Sci 1:e18 https://doi.org/10.7717/peerj-cs.18

Gururangan S, Marasovic A, Swayamdipta S, Lo K, Beltagy I, Downey D, Smith NA (2020) Don't stop pretraining: Adapt language models to domains and tasks. arXiv preprint arXiv:2004.10964

Guzman E, Bruegge B (2013) Towards emotional awareness in software development teams. In: Proceedings of the 2013 9th joint meeting on foundations of software engineering, pp 671–674 https://doi.org/10.1145/2491411.2494578

Guzman E, Azócar D, Li Y (2014) Sentiment analysis of commit comments in github: An empirical study. In: Proceedings of the 11th working conference on mining software repositories, ACM, pp 352–355 https://doi.org/10.1145/2597073.2597118

Hasan MA, Das S, Anjum A, Alam F, Anjum A, Sarker A, Noori SRH (2024) Zero- and few-shot prompting with llms: A comparative study with fine-tuned models for bangla sentiment analysis. arXiv preprint arXiv:2308.10783v2

Hata H, Novielli N, Baltes S, Kula RG, Treude C (2022) Github discussions: An exploratory study of early adoption. Empir Softw Eng 27(1):3 https://doi.org/10.1007/s10664-021-10058-6

Herrmann M, Klünder J (2021) From textual to verbal communication: Towards applying sentiment analysis to a software project meeting. In: 2021 Fourth International Workshop on Affective Computing for Requirements Engineering (AffectRE). IEEE https://doi.org/10.1109/REW53955.2021.00065

Imran MM, Chatterjee P, Damevski K (2024) Uncovering the causes of emotions in software developer communication using zero-shot llms. In: Proceedings of the IEEE/ACM 46th international conference on software engineering, pp 1–13 https://doi.org/10.1145/3597503.3639223

Islam MR, Zibran MF (2017) Leveraging automated sentiment analysis in software engineering. In: 2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR), pp 203–214. https://doi.org/10.1109/MSR.2017.9

Islam MR, Zibran MF (2018) Sentistrength-se: Exploiting domain specificity for improved sentiment analysis in software engineering text. J Syst Softw 145:125–146 https://doi.org/10.1016/j.jss.2018.08.030

Kalliamvakou E, Damian D, Blincoe K, Singer L, German DM (2015) Open source-style collaborative development practices in commercial projects using github. In: 2015 IEEE/ACM 37th IEEE international conference on software engineering, IEEE, vol 1, pp 574–585 https://doi.org/10.1109/ICSE.2015.74

Kolchyna O, Souza TT, Treleaven P, Aste T (2015) Twitter sentiment analysis: Lexicon method, machine learning method and their combination. arXiv preprint arXiv:1507.00955

Li Z, Peng B, He P, Yan X (2023) Evaluating the instruction-following robustness of large language models to prompt injection. arXiv preprint arXiv:2308.10819

Lin B, Zampetti F, Bavota G, Di Penta M, Lanza M, Oliveto R (2018) Sentiment analysis for software engineering: How far can we go? In: Proceedings of the 40th International Conference on Software Engineering, Association for Computing Machinery, New York, NY, USA, ICSE '18, pp 94–104. https://doi.org/10.1145/3180155.3180195

Liu B (2012) Sentiment analysis and opinion mining. Springer Nature https://doi.org/10.1162/COLI_r_00186

Mannan UA, Ahmed I, Jensen C, Sarma A (2020) On the relationship between design discussions and design quality: A case study of apache projects. In: Proceedings of the 28th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering, pp 543–553 https://doi.org/10.1145/3368089.3409707

Marvin G, Hellen N, Jjingo D, Nakatumba-Nabende J (2023) Prompt engineering in large language models. In: International conference on data intelligence and cognitive informatics, Springer, pp 387–402 https://doi.org/10.1007/978-981-99-7962-2_30

Niimi J (2024) Dynamic sentiment analysis with local large language models using majority voting: A study on factors affecting restaurant evaluation. arXiv preprint arXiv:2407.13069

Novielli N, Calefato F, Dongiovanni D, Girardi D, Lanubile F (2020) Can we use se-specific sentiment analysis tools in a cross-platform setting? In: Proceedings of the 17th international conference on mining software repositories, association for computing machinery, New York, NY, USA, MSR '20, pp 158–168. https://doi.org/10.1145/3379597.3387446

Novielli N, Girardi D, Lanubile F (2018) A benchmark study on sentiment analysis for software engineering research. In: Proceedings of the 15th international conference on mining software repositories, pp 364–375 https://doi.org/10.1145/3196398.3196403

Obaidi M, Herrmann M, Schneider K, Klünder J (2025a) Towards trustworthy sentiment analysis in software engineering: Dataset characteristics and tool selection. In: 2025 IEEE 33rd International Requirements Engineering Conference Workshops (REW), IEEE, pp 538–547 https://doi.org/10.1109/REW66121.2025.00080

Obaidi M, Herrmann M, Schmid E, Ochsner R, Schneider K, Klünder J (2025b) A german gold-standard dataset for sentiment analysis in software engineering. In: 2025 IEEE 33rd International Requirements Engineering Conference Workshops (REW), IEEE, pp 107–114 https://doi.org/10.1109/REW66121.2025.00018

Obaidi M, Nagel L, Specht A, Klünder J (2022) Sentiment analysis tools in software engineering: A systematic mapping study. Inform Softw Technol 107018 https://doi.org/10.1016/j.infsof.2022.107018

Ortu M, Murgia A, Destefanis G, Tourani P, Tonelli R, Marchesi M, Adams B (2016) The emotional side of software developers in jira. In: Proceedings of the 13th International Conference on Mining Software Repositories, Association for Computing Machinery, New York, NY, USA, MSR '16, pp 480–483. https://doi.org/10.1145/2901739.2903505, https://doi.org/10.1145/2901739.2903505

Pang B, Lee L et al (2008) Opinion mining and sentiment analysis. Found Trends® Inf Retriev 2(1–2):1–135

Rahman MM, Roy CK (2014) An insight into the pull requests of github. In: 11th working conference on mining software repositories, pp 364–367 https://doi.org/10.1145/2597073.2597121

Sayago-Heredia J, Chango G, Pérez-Castillo R, Piattini M (2022) Exploring the impact of toxic comments in code quality. In: ENASE, pp 335–343 https://doi.org/10.5220/0011039700003176

Shafikuzzaman M, Islam MR, Rolli AC, Akhter S, Seliya N (2024) An empirical evaluation of the zero-shot, few-shot, and traditional fine-tuning based pretrained language models for sentiment analysis in software engineering. IEEE Access https://doi.org/10.1109/ACCESS.2024.3439450

Shaver P, Schwartz J, Kirson D, O'connor C (1987) Emotion knowledge: further exploration of a prototype approach. J Pers Soc Psychol 52(6):1061 https://doi.org/10.1037/0022-3514.52.6.1061

Tantisuwankul J, Nugroho YS, Kula RG, Hata H, Rungsawang A, Leelaprute P, Matsumoto K (2019) A topological analysis of communication channels for knowledge sharing in contemporary github projects. J Syst Softw 158:110416 https://doi.org/10.1016/j.jss.2019.110416

Thelwall M, Buckley K, Paltoglou G (2012) Sentiment strength detection for the social web. J Am Soc Inform Sci Technol 63(1):163–17. https://doi.org/10.1002/asi.21662

Tsay J, Dabbish L, Herbsleb J (2014a) Influence of social and technical factors for evaluating contribution in github. In: Proceedings of the 36th international conference on Software engineering, pp 356–366 https://doi.org/10.1145/2568225.2568315

Tsay J, Dabbish L, Herbsleb J (2014b) Let's talk about it: Evaluating contributions through discussion in github. In: Proceedings of the 22nd ACM SIGSOFT international symposium on foundations of software engineering, association for computing machinery, New York, NY, USA, FSE 2014, pp 144–154. https://doi.org/10.1145/2635868.2635882

Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser Ł, Polosukhin I (2017) Attention is all you need. Adv Neural Inf Process Syst 30

Viviani G, Famelis M, Xia X, Janik-Jones C, Murphy GC (2019) Locating latent design information in developer discussions: A study on pull requests. IEEE Trans Software Eng 47(7):1402–1413 https://doi.org/10.1109/TSE.2019.2924006

Wei J, Wang X, Schuurmans D, Bosma M, Ichter B, Xia F, Chi EH, Le QV, Zhou D (2022) Chain of thought prompting elicits reasoning in large language models. arXiv preprint arXiv:2201.11903

Wohlin C, Runeson P, Höst M, Ohlsson M, Regnell B, Wesslén A (2012) Experimentation in software engineering, 1st edn. Springer Science & Business Media https://doi.org/10.1007/978-3-642-29044-2

Xing F (2024) Designing heterogeneous llm agents for financial sentiment analysis. arXiv preprint arXiv:2401.05799

Zhan T, Shi C, Shi Y, Li H, Lin Y (2024) Optimization techniques for sentiment analysis based on llm (gpt-3). arXiv preprint arXiv:2405.09770

Zhang T, Xu B, Thung F, Haryono SA, Lo D, Jiang L (2020) Sentiment analysis for software engineering: How far can pre-trained transformer models go? In: 2020 IEEE International Conference on Software Maintenance and Evolution (ICSME), IEEE, pp 70–80 https://doi.org/10.1109/ICSME46990.2020.00017

Zhang X, Yu Y, Gousios G, Rastogi A (2022) Pull request decisions explained: An empirical overview. IEEE Trans Software Eng 49(2):849–871 https://doi.org/10.1109/TSE.2022.3165056

Zhang T, Irsan IC, Thung F, Lo D (2025) Revisiting sentiment analysis for software engineering in the era of large language models. IEEE Trans Software Eng 34(3): 1-30 https://doi.org/10.1145/3697009

Zhou Y, Muresanu AI, Han Z, Paster K, Pitis S, Chan H, Ba J (2022) Large language models are human-level prompt engineers. arXiv preprint arXiv:2211.01910

Zhu K, Wang J, Zhou J, Wang Z, Chen H, Wang Y, Yang L, Ye W, Zhang Y, Gong N et al (2023) Promptrobust: Towards evaluating the robustness of large language models on adversarial prompts. In: Proceedings of the 1st ACM workshop on large AI systems and models with privacy and safety analysis, pp 57–68 https://doi.org/10.1145/3689217.3690621

Authors and Affiliations

Daniel Coutinho[1] · Breno Braga[1] · Theo Canuto[1] · Juliana Alves Pereira[1] · Wesley K. G. Assunção[2] · Igor Steinmacher[3] · Marco Gerosa[3] · Alessandro Garcia[1]

✉ Daniel Coutinho
dcoutinho@inf.puc-rio.br

Breno Braga
brenonevs@aise.inf.puc-rio.br

Theo Canuto
theo@aise.inf.puc-rio.br

Juliana Alves Pereira
juliana@inf.puc-rio.br

Wesley K. G. Assunção
wguezas@ncsu.edu

Igor Steinmacher
Igor.Steinmacher@nau.edu

Marco Gerosa
Marco.Gerosa@nau.edu

Alessandro Garcia
afgarcia@inf.puc-rio.br

1 Pontifical Catholic University of Rio de Janeiro (PUC-Rio), Rio de Janeiro, Brazil

2 North Carolina State University, Raleigh, NC, United States

3 Northern Arizona University (NAU), Flagstaff, AZ, United States